字符串

Morning Flower and Evening Oath

XCPC vector 模板分册

建设稿：验证状态与适用边界见各条目及仓库 verification/。

目录

使用约定 iii

第一章 字符串哈希 1
1.1 子串与反串双哈希 . . . 1

第二章 前缀函数与 KMP 算法 5
2.1 前缀函数 . . . 5
2.2 KMP 匹配 . . . 7
2.3 顺序同构匹配 . . . 9

第三章 Z 函数（扩展 KMP） 11
3.1 Z 函数 . . . 11
3.2 相邻十进制好数之和 . . . 12
3.3 两串扩展 KMP . . . 14

第四章 AC 自动机 16
4.1 AC 自动机出现次数 . . . 16
4.2 AC 自动机最短后缀匹配长度 . . . 22
4.3 AC 自动机带权匹配 . . . 24
4.4 动态 AC：二进制分组重建 . . . 26

第五章 后缀数组 (SA) 28
5.1 后缀数组简介 . . . 28
5.1.1 线性后缀数组（SA-IS） . . . 28
5.1.2 倍增后缀数组与 LCP . . . 32
5.1.3 后缀 LCP 与子串比较 . . . 35
5.1.4 两个前缀的最长公共后缀 . . . 38
5.1.5 重复子串 AA 的起止位置计数 . . . 38
5.1.6 后缀数组：DC3 线性构造 . . . 40

第六章 后缀自动机 (SAM) 45
6.1 后缀自动机 . . . 45
6.2 SAM 字典序子串查询 . . . 50
6.3 SAM 最长公共子串 . . . 53

第七章 广义后缀自动机 55
7.1 广义后缀自动机：Trie BFS 构建 . . . 55
7.2 指定状态在线扩展 SAM . . . 62
7.3 SAM 文档频率与阈值后缀 . . . 65

第八章 Manacher 70
8.1 Manacher 回文半径 . . . 70

第九章 回文树 73
9.1 回文树 PAM：回文后缀与出现次数 . . . 73

第十章 最小表示法 **76**
10.1 最小循环表示 76

第十一章 Lyndon 分解 **78**
11.1 Lyndon 分解：Duval 78

第十二章 配套知识与验证范围 **80**
12.1 回文半径与在线回文计数 80
12.2 整数后缀数组的边界与 SAM 次数 80
12.3 广义 SAM 点数与后缀语言 80

第一部分 跨类依赖 **82**

附录 A 数论 **83**
A.1 模算术简介 83
A.1.1 静态模整数 83
A.2 卢卡斯定理 86
A.2.1 Lucas：小素数模的大下标组合数 86

附录 B 组合数学 **88**
B.1 排列组合 88
B.1.1 阶乘组合数 88

附录 C 树状数组 **90**
C.1 树状数组第 k 小与组合依赖 90

使用约定

C++20，默认包含 bits/stdc++.h 与 cassert，并使用 std 命名空间。下标、区间及数值范围以各条目说明为准。DFS 保持递归，需满足题目栈空间条件。

本册主目录沿用 OI Wiki 官方导航的大、中、小层级，各页下列本库实现。PBDS 与 GNU rope 按队内约定纳入平衡树。组合条目与相关挂靠仍见仓库分类审计。跨类引用的已收录组件附在依赖部分；源头文件中的依赖仍须一并检查。

本册尚非全量完成稿；传统版评测仅为历史档案，当前只维护 vector 代码。

第一章　字符串哈希

1.1　子串与反串双哈希

StringHash(s, base)，n<INT_MAX，两个底数分别在 [257,mod-2]，所有比较对象共享同组底数。get(l, r, rev=false) 取 0-based 半开区间；rev=true 反转原区间，空区间返回 {0,0}。join(a, b, len_b) 拼接，要求规范余数、同底数及 0≤len_b≤n，可拼出长度超过 n 的哈希。构造 O(n)，查询/拼接 O(1)，约 24(n+1) 字节；按 unsigned char 加 1 编码。判等必须先比长度；双哈希仍可能碰撞。理想独立均匀底数下，固定同长不同串的碰撞上界为两个 min(1,(L-1)/(mod[j]-258)) 之积，多次比较用联合界；可预测种子和适应性输入不享有该保证。需确定性时用直接比较或 SuffixArray + SuffixLCP；回文可用 Manacher。成员只读。

```
struct StringHash
{
    using H = array<int, 2>;
    static constexpr H mod = {1000000007, 1000000009};
    int n;
    H base;
    vector<H> pw, pre, rev;

    StringHash(const string &s, H b)
        : n(s.size()), base(b), pw(n + 1), pre(n + 1), rev(n + 1)
    {
        pw[0] = {1, 1};
        for (int j = 0; j < 2; j++)
        {
            assert(257 <= base[j] && base[j] <= mod[j] - 2);
            for (int i = 0; i < n; i++)
            {
                int x = (unsigned char)s[i] + 1;
                int y = (unsigned char)s[n - 1 - i] + 1;
                pw[i + 1][j] = 1LL * pw[i][j] * base[j] % mod[j];
                pre[i + 1][j] = (1LL * pre[i][j] * base[j] + x) % mod[j];
                rev[i + 1][j] = (1LL * rev[i][j] * base[j] + y) % mod[j];
            }
        }
    }
```

子串、反串与拼接（接上页同一结构体）：

```
    H get(int l, int r, bool reversed = false) const
    {
        assert(0 <= l && l <= r && r <= n);
        if (reversed)
        {
            int x = n - r;
            r = n - l;
            l = x;
        }
        const auto &h = reversed ? rev : pre;
        H ans;
        for (int j = 0; j < 2; j++)
        {
            long long x = h[r][j] - 1LL * h[l][j] * pw[r - l][j] % mod[j];
            ans[j] = (x + mod[j]) % mod[j];
        }
        return ans;
    }

    H join(H a, H b, int len_b) const
    {
        assert(0 <= len_b && len_b <= n);
        for (int j = 0; j < 2; j++)
        {
            a[j] = (1LL * a[j] * pw[len_b][j] + b[j]) % mod[j];
        }
        return a;
    }
};
```

模板题使用：P3370 【模板】字符串哈希

题意：输入 N 个非空字母数字串，输出不同串数。随机底数全局共享，按长度与双哈希去重；概率判等，AC 不证明零碰撞。

所需模板：StringHash（第 1 页，1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3370

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    mt19937 rng(random_device{}());
    StringHash::H base;
    for (int j = 0; j < 2; j++)
    {
        base[j] = uniform_int_distribution<int>(257, StringHash::mod[j] - 2)(rng);
    }
    int n;
    cin >> n;
    set<pair<int, StringHash::H>> seen;
    while (n--)
    {
        string s;
        cin >> s;
        StringHash h(s, base);
        seen.insert({int(s.size()), h.get(0, s.size())});
    }
    cout << seen.size() << '\n';
}
```

接口用法演示：API 子串、反串与拼接

演示约定：首行字符串可空，再输入两分量底数、q；每次 l r a b 为原串半开区间，输出 [l,r) 正向/反向、[l,r)+[a,b) 的双哈希。底数显式输入便于重现，不是固定底数推荐。

所需模板：StringHash（第 1 页，1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/string/hash/

```
int main()
{
    string s;
    getline(cin, s);
    StringHash::H base;
    cin >> base[0] >> base[1];
    StringHash h(s, base);
    int q;
    cin >> q;
    while (q--)
    {
        int l, r, a, b;
        cin >> l >> r >> a >> b;
        auto x = h.get(l, r);
        auto y = h.get(l, r, true);
        auto z = h.join(x, h.get(a, b), b - a);
        cout << x[0] << ' ' << x[1] << '\n';
        cout << y[0] << ' ' << y[1] << '\n';
        cout << z[0] << ' ' << z[1] << '\n';
    }
}
```

第二章　前缀函数与 KMP 算法

2.1　前缀函数

prefix_function(s) 返回每个前缀的最长真 border 长度，0-based；空串返回空数组。O(n)，无状态，直接抄写函数。

```
vector<int> prefix_function(const string &s)
{
    int n = (int)s.size();
    vector<int> p(n);
    for (int i = 1; i < n; i++)
    {
        int j = p[i - 1];
        while (j && s[i] != s[j]) j = p[j - 1];
        p[i] = j + (s[i] == s[j]);
    }
    return p;
}
```

联合使用示例见第 8 页（Luogu P3375），其中直接调用本模板。

接口用法演示：优化 KMP 失败表 nextval 与重叠匹配

演示约定：多组直到 EOF：输入文本长度 n≥0、非空模式长度 1≤m<INT_MAX，随后两串各自的 0..255 字节整数。每组输出 nextval[0..m]、匹配数量、0-based 起点三行。对 i<m 跳过下一个模式字符仍相同的失败状态；nextval[m] 必须取普通最长 border，保留重叠。无需尾零，支持零字节。O(n+m) 时间，O(m+ 答案数) 额外空间；来源接口演示，不是线上模板题。

所需模板：prefix_function（第 5 页，2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/kuangbin/ACM-ICPC

```cpp
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    while (cin >> n >> m)
    {
        assert(0 <= n && 1 <= m && m < INT_MAX);
        string s(n, 0), t(m, 0);
        for (char &c : s)
        {
            int x;
            cin >> x;
            assert(0 <= x && x < 256);
            c = x;
        }
        for (char &c : t)
        {
            int x;
            cin >> x;
            assert(0 <= x && x < 256);
            c = x;
        }
        auto p = prefix_function(t);
        vector<int> nextval(m + 1, -1);
        for (int i = 1; i < m; i++)
        {
            int j = p[i - 1];
            nextval[i] = t[i] == t[j] ? nextval[j] : j;
        }
        nextval[m] = p[m - 1];
        for (int x : nextval)
            cout << x << ' ';
        cout << '\n';
```

继续处理操作（接上页同一 main）：

```
        vector<int> positions;
        int j = 0;
        for (int i = 0; i < n; i++)
        {
            while (j != -1 && s[i] != t[j])
                j = nextval[j];
            j++;
            if (j == m)
            {
                positions.push_back(i - m + 1);
                j = nextval[m];
            }
        }
        cout << positions.size() << '\n';
        for (int x : positions)
            cout << x << ' ';
        cout << '\n';
    }
    return 0;
}
```

2.2 KMP 匹配

代码依赖：prefix_function（第 5 页，2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

kmp_match(s,t) 返回非空模式串 t 在 s 中的所有起点（0-based），允许重叠。需要同章前缀函数。O(|s|+|t|)。

```
vector<int> kmp_match(const string &s, const string &t)
{
    assert(!t.empty());
    auto p = prefix_function(t);
    vector<int> ans;
    for (int i = 0, j = 0; i < (int)s.size(); i++)
    {
        while (j && s[i] != t[j]) j = p[j - 1];
        j += (s[i] == t[j]);
        if (j == (int)t.size())
        {
            ans.push_back(i - j + 1);
            j = p[j - 1];
        }
    }
    return ans;
}
```

模板题使用：Luogu P3375

题意：求模式串 t 在文本 s 中所有出现的位置，包含重叠匹配；模板返回 0-based 位置，输出加 1。最后一行直接输出 prefix_function(t)，它是模式串各前缀的最长真 border 长度。题目输入为非空大写串，函数按字符相等比较，不限于小写。

所需模板：prefix_function（第 5 页，2.1 节）、kmp_match（第 7 页，2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3375

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s, t;
    cin >> s >> t;
    for (int p : kmp_match(s, t)) cout << p + 1 << '\n';
    for (int x : prefix_function(t)) cout << x << ' ';
    cout << '\n';
}
```

相关应用：大小和相等关系的顺序同构匹配；KMP 推广。

2.3 顺序同构匹配

order_match(s,t) 接收 vector<long long>，t 非空；返回所有 0-based 起点，含重叠。匹配要求任意两位置的大小及相等关系一致，数值本身可不同。支持完整 signed64，仅比较不作差；输入不变，长度均 <INT_MAX。模式预处理记录之前的相等、前驱和后继位置，再用相对下标检查 KMP 扩展。O(|t|log(|t|+1)+|s|) 时间，O(|t|+答案数) 空间。P6080 为应用题，不作为正式模板题覆盖。

```
// Match all pairwise <, =, > relations; t must be nonempty.
vector<int> order_match(const vector<long long> &s, const vector<long long> &t)
{
    assert(!t.empty());
    int n = s.size(), m = t.size();
    vector<int> eq(m, -1), lo(m, -1), hi(m, -1), p(m), ans;
    map<long long, int> last;
    for (int j = 0; j < m; j++)
    {
        auto it = last.lower_bound(t[j]);
        if (it != last.end() && it->first == t[j])
            eq[j] = it->second;
        else
        {
            if (it != last.end()) hi[j] = it->second;
            if (it != last.begin()) lo[j] = prev(it)->second;
        }
        last[t[j]] = j;
    }
    auto fits = [&](const vector<long long> &a, int i, int j)
    {
        int start = i - j;
        if (eq[j] != -1) return a[i] == a[start + eq[j]];
        if (lo[j] != -1 && a[i] <= a[start + lo[j]]) return false;
        if (hi[j] != -1 && a[i] >= a[start + hi[j]]) return false;
        return true;
    };
    for (int i = 1, j = 0; i < m; i++)
    {
        while (j && !fits(t, i, j)) j = p[j - 1];
        if (fits(t, i, j)) j++;
        p[i] = j;
    }
    for (int i = 0, j = 0; i < n; i++)
    {
        while (j && !fits(s, i, j)) j = p[j - 1];
        if (fits(s, i, j)) j++;
        if (j == m)
        {
            ans.push_back(i - m + 1);
            j = p[j - 1];
        }
    }
    return ans;
}
```

应用题补充：Luogu P6080 / Cow Patterns

题意：N≤100000、K≤25000、S≤25，值在 1..S；输出匹配的个数与升序 1-based 起点，逐行输出。保持相等和严格大小关系，允许重叠；S 无需用于算法。应用补充，未在线 AC。

所需模板：order_match（第 9 页，2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6080

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, alphabet;
    cin >> n >> m >> alphabet;
    vector<long long> s(n), t(m);
    for (auto &x : s) cin >> x;
    for (auto &x : t) cin >> x;
    auto ans = order_match(s, t);
    cout << ans.size() << '\n';
    for (int x : ans) cout << x + 1 << '\n';
}
```

第三章　Z 函数（扩展 KMP）

3.1　Z 函数

z_function(s) 返回后缀 s[i..] 与原串的 LCP，约定 z[0]=n，空串返回空数组。O(n)。

```
vector<int> z_function(const string &s)
{
    int n = (int)s.size();
    vector<int> a(n);
    if (n) a[0] = n;
    for (int i = 1, l = 0, r = 0; i < n; i++)
    {
        if (i < r) a[i] = min(r - i, a[i - l]);
        while (i + a[i] < n && s[a[i]] == s[i + a[i]]) ++a[i];
        if (i + a[i] > r)
        {
            l = i;
            r = i + a[i];
        }
    }
    return a;
}
```

模板题使用：Library Checker zalgorithm

题意：给非空字符串，对每个起点求其后缀与原串的最长公共前缀长度。直接输出 Z 数组，z[0] 等于串长。

所需模板：z_function（第 11 页，3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/zalgorithm

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    auto z = z_function(s);
    for (int i = 0; i < (int)z.size(); i++)
        cout << z[i] << (i + 1 == (int)z.size() ? '\n' : ' ');
}
```

3.2 相邻十进制好数之和

代码依赖：exkmp（第 14 页，3.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

good_number_sum(s,x,mod=2^61-1) 返回 array<int,3>{l,k,r}，表示 s[l,k) 与 s[k,r) 这两个相邻非空子串的十进制数值之和为 x；无解返回 {-1,-1,-1}。s 和非空 x 都只能含 1..9，不能含 0；|s|≤500000，|x|≤200000。允许空 s 并返回无解。模数 1≤mod<2^61，不要求素数；它仅影响筛选效率，不影响正确性。

用 exkmp 缩减候选长度，再用底 10 数值余数筛选，命中后逐位验证实际加法与进位，避免哈希碰撞输出错误答案。不是普通 StringHash 字符哈希相加；不能取消最终验证。返回任意合法方案，不保证字典序最小。

令 n=|s|、m=|x|，生成候选 O(n+m)，最多检查 O(n) 个候选；若 C 个候选通过余数筛选，总时间 O(n+m+C*m)，最坏 O(n*m+n+m)，空间 O(n+m)。默认固定大模数通常筛选较强，但不宣称对抗输入的线性时间；mod=1 允许强制所有候选碰撞以检验正确性。128 位乘积精确取模，余数加法不溢出 unsigned64。依赖 exkmp 及 z_function。CF1598G 完整协议、原稿越界反例和大输入本地验证，无新在线 AC。

```cpp
// Return adjacent half-open ranges [l,k), [k,r), or {-1,-1,-1}.
array<int, 3> good_number_sum(const string &s, const string &x,
                              unsigned long long mod = (1ULL << 61) - 1)
{
    using U = unsigned long long;
    using W = __uint128_t;
    assert(s.size() <= 500000 && !x.empty() && x.size() <= 200000);
    assert(1 <= mod && mod < (1ULL << 61));
    for (char c : s)
        assert('1' <= c && c <= '9');
    for (char c : x)
        assert('1' <= c && c <= '9');
    int n = s.size(), m = x.size();
    vector<U> h(n + 1), power(n + 1);
    power[0] = 1 % mod;
    for (int i = 0; i < n; i++)
    {
        h[i + 1] = (W(h[i]) * 10 + s[i] - '0') % mod;
        power[i + 1] = W(power[i]) * 10 % mod;
    }
    U target = 0;
    for (char c : x)
        target = (W(target) * 10 + c - '0') % mod;
    auto remainder = [&](int l, int r) -> U
    {
        U removed = W(h[l]) * power[r - l] % mod;
        return (h[r] + mod - removed) % mod;
    };
```

接上页同一函数:

```
    auto valid = [&](int l, int k, int r)
    {
        if (l < 0 || l >= k || k >= r || r > n)
            return false;
        if ((remainder(l, k) + remainder(k, r)) % mod != target)
            return false;
        int a = k - 1, b = r - 1, carry = 0;
        for (int i = m - 1; i >= 0; i--)
        {
            int digit = carry;
            if (a >= l)
                digit += s[a--] - '0';
            if (b >= k)
                digit += s[b--] - '0';
            if (digit % 10 != x[i] - '0')
                return false;
            carry = digit / 10;
        }
        return a < l && b < k && carry == 0;
    };
    if (m > 1)
        for (int l = 0; l + 2 * (m - 1) <= n; l++)
        {
            int k = l + m - 1, r = k + m - 1;
            if (valid(l, k, r))
                return {l, k, r};
        }
    auto [z, lcp] = exkmp(s, x);
    for (int i = 0; i + m <= n; i++)
        for (int len : {m - lcp[i], m - lcp[i] - 1})
        {
            if (len <= 0)
                continue;
            if (valid(i - len, i, i + m))
                return {i - len, i, i + m};
            if (valid(i, i + m, i + m + len))
                return {i, i + m, i + m + len};
        }
    return {-1, -1, -1};
}
```

应用题补充：CF1598G：The Sum of Good Numbers

题意：输入仅含 1..9 的字符串 s 和目标整数 x，题面保证有解。输出相邻两个子串的一基闭区间，使对应整数之和为 x；直接把返回 {l,k,r} 写成 [l+1,k] 和 [k+1,r]。余数碰撞后仍逐位验证，避免错误方案。此驱动另外以-1表示无解，但正式题目不出现无解。

所需模板：z_function（第 11 页，3.1 节）、exkmp（第 14 页，3.3 节）、good_number_sum（第 12 页，3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/contest/1598/problem/G

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s, x;
    cin >> s >> x;
    auto [l, k, r] = good_number_sum(s, x);
    if (l == -1)
        cout << -1 << '\n';
    else
    {
        cout << l + 1 << ' ' << k << '\n';
        cout << k + 1 << ' ' << r << '\n';
    }
}
```

3.3 两串扩展 KMP

代码依赖：z_function（第 11 页，3.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

exkmp(s,t) 返回 {z,p}：z 为模式 t 的 Z 数组，z[0]=|t|；p[i] 是文本后缀 s[i..] 与 t 的 LCP 长度，均 0-based。空串合法，t 空时 p 全 0，s 空时 p 为空。任意字节相等比较，含零字节与高位字节；不拼接分隔符、不拷贝输入。需要 z_function。两串长度均须 <INT_MAX；O(|s|+|t|) 时间，两个输出 vector 共 O(|s|+|t|) 空间，返回时移动缓冲区。维护右端最远的匹配区间 [l,r)，区间内利用模式自身 Z 值再向右扩展。P5410 的 1-based 权值乘法应先提升到 long long。

```
// Return {Z(t), LCP(s[i..], t) for every i}; empty strings are allowed.
pair<vector<int>, vector<int>> exkmp(const string &s, const string &t)
{
    int n = s.size(), m = t.size();
    auto z = z_function(t);
    vector<int> p(n);
    for (int i = 0, l = 0, r = 0; i < n; i++)
    {
        if (i < r) p[i] = min(r - i, z[i - l]);
        while (p[i] < m && p[i] < n - i && s[i + p[i]] == t[p[i]])
            p[i]++;
        if (i + p[i] > r)
        {
            l = i;
            r = i + p[i];
        }
    }
    return {move(z), move(p)};
}
```

模板题使用：Luogu P5410 / 扩展 KMP

题意：输入小写串 a、b，长度均 1..20000000。求 b 的 Z 数组与 b 对 a 各后缀的 LCP 数组，各输出 xor((i+1)*(长度 +1))。z[0]=|b|；乘法先提升 long long。直接求两串 LCP，不拼接整串。

所需模板：z_function（第 11 页，3.1 节）、exkmp（第 14 页，3.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5410

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string a, b;
    cin >> a >> b;
    auto [z, p] = exkmp(a, b);
    long long x = 0, y = 0;
    for (int i = 0; i < (int)z.size(); i++)
        x ^= 1LL * (i + 1) * (z[i] + 1);
    for (int i = 0; i < (int)p.size(); i++)
        y ^= 1LL * (i + 1) * (p[i] + 1);
    cout << x << '\n' << y << '\n';
}
```

第四章　AC 自动机

4.1　AC 自动机出现次数

add(s)/count(s) 默认小写 string；add(s,offset)/count(s,offset) 接受 string 或 vector<int>，每个元素减 offset 须在 0..25，同一自动机使用一致编码。字符串字面量请先转 string 再传双参数入口。add 返回终态，支持空模式返回根 0；count 返回逐状态重叠出现次数，根恒为文本长度 +1。全部插入后 build 一次，随后只读且可重复查询；重复模式共享状态。a[u].len 为 Trie 深度，go/fail 与 order 可读，order 不含根。size 用 a.size()，重置赋新对象。构建 O(26V+L)，查询 O(|text|+V)，空间 O(26V)。状态数、深度须在 int 内。非空旧用法不变；旧根计数语义需减 1。出现过的模式编号数仍按每个终点分别判断 cnt>0。

```
struct AhoCorasick
{
    struct Node
    {
        array<int, 26> go{};
        int fail = 0, len = 0;
    };

    vector<Node> a{Node{}};
    vector<int> order;
    bool built = false;

    template <class S>
    int add(const S &s, int offset)
    {
        assert(!built);
        int u = 0;
        for (auto ch : s)
        {
            long long c = (long long)ch - offset;
            assert(0 <= c && c < 26);
            if (!a[u].go[c])
            {
                int v = (int)a.size();
                a[u].go[c] = v;
                a.push_back(Node{});
                a[v].len = a[u].len + 1;
            }
            u = a[u].go[c];
        }
        return u;
    }

    int add(const string &s)
    {
        return add(s, 'a');
    }
```

构建失败链接与统计出现次数（接上页同一结构体）：

```
    void build()
    {
        assert(!built);
        built = true;
        queue<int> q;
        for (int v : a[0].go)
            if (v) q.push(v);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            order.push_back(u);
            for (int c = 0; c < 26; c++)
            {
                int v = a[u].go[c];
                if (v)
                {
                    a[v].fail = a[a[u].fail].go[c];
                    q.push(v);
                }
                else
                    a[u].go[c] = a[a[u].fail].go[c];
            }
        }
    }

    template <class S>
    vector<long long> count(const S &s, int offset) const
    {
        assert(built);
        vector<long long> ans(a.size());
        ans[0] = 1;
        int u = 0;
        for (auto ch : s)
        {
            long long c = (long long)ch - offset;
            assert(0 <= c && c < 26);
            u = a[u].go[c];
            ++ans[u];
        }
        for (int i = (int)order.size() - 1; i >= 0; i--)
            ans[a[order[i]].fail] += ans[order[i]];
        return ans;
    }

    vector<long long> count(const string &s) const
    {
        return count(s, 'a');
    }
};
```

模板题使用：Luogu P5357

题意：给多个小写字母模式和文本，输出每个模式的出现次数，允许重叠。先保存 add 返回的终止状态，全部插入后 build，再 count 文本并按状态取值；重复模式共享状态，仍按输入顺序输出。

所需模板：AhoCorasick（第 16 页，4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5357

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    AhoCorasick ac;
    vector<int> terminal(n);
    for (int &id : terminal)
    {
        string pattern;
        cin >> pattern;
        id = ac.add(pattern);
    }
    ac.build();
    string text;
    cin >> text;
    auto count = ac.count(text);
    for (int id : terminal) cout << count[id] << '\n';
}
```

模板题使用：Luogu P3808 / AC 自动机简单版

题意：给非空小写模式串和文本，求出现过的模式编号数；两个相同字符串但编号不同要分别计数，同一编号多次出现只算一次。模式数、模式总长和文本长度均至 1000000。保存每次 add 返回的终点，count 后逐编号判断是否为正；不清空终态，因此可重复查询。

所需模板：AhoCorasick（第 16 页，4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3808

```cpp
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    AhoCorasick ac;
    vector<int> end(n);
    for (int &u : end)
    {
        string s;
        cin >> s;
        u = ac.add(s);
    }
    ac.build();
    string t;
    cin >> t;
    auto cnt = ac.count(t);
    int ans = 0;
    for (int u : end) ans += cnt[u] > 0;
    cout << ans << '\n';
}
```

接口用法演示：整数序列 AC 与深度（接口演示）

演示约定：输入 n,q,offset，每条模式/查询给出长度和整数序列。各元素减 offset 必须为 0..25，允许空序列。先输出各模式 Trie 深度，再逐查询输出各模式出现次数；空模式贡献文本长度 +1。

所需模板：AhoCorasick（第 16 页，4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/string/ac-automaton/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q, offset;
    cin >> n >> q >> offset;
    AhoCorasick ac;
    vector<int> end(n);
    for (int &u : end)
    {
        int m;
        cin >> m;
        vector<int> s(m);
        for (int &x : s) cin >> x;
        u = ac.add(s, offset);
    }
    ac.build();
    for (int u : end) cout << ac.a[u].len << ' ';
    cout << '\n';
    while (q--)
    {
        int m;
        cin >> m;
        vector<int> s(m);
        for (int &x : s) cin >> x;
        auto cnt = ac.count(s, offset);
        for (int u : end) cout << cnt[u] << ' ';
        cout << '\n';
    }
}
```

接口用法演示：自定义字符偏移 AC（接口演示）

演示约定：输入 n,q 及字母表起始字符 offset，再输入 n 个模式串和 q 个文本，均用双引号包围以支持空串。字符减 offset 为 0..25。逐查询输出所有模式的重叠出现次数；例如 offset 为 A 可处理大写字母。

所需模板：AhoCorasick（第 16 页，4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/string/ac-automaton/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    char offset;
    cin >> n >> q >> offset;
    AhoCorasick ac;
    vector<int> end(n);
    for (int &u : end)
    {
        string s;
        cin >> quoted(s);
        u = ac.add(s, offset);
    }
    ac.build();
    while (q--)
    {
        string s;
        cin >> quoted(s);
        auto cnt = ac.count(s, offset);
        for (int u : end) cout << cnt[u] << ' ';
        cout << '\n';
    }
}
```

4.2 AC 自动机最短后缀匹配长度

代码依赖：AhoCorasick（第 16 页，4.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

先用 AhoCorasick.add(s) 得到终止节点，记录 (节点, 模式长度)，再 build 后调用 ac_shortest(ac,ends)。返回每个状态沿 fail 链对应的最短已登记模式长度，无模式为 INT_MAX。登记长度须真实且在 1..INT_MAX-1；重复模式、部分模式集合、空集合均可，未登记的模式不会参与最小值。按 ac.order 的 BFS 顺序传播，父 fail 状态先完成；O(状态数 + 登记数) 时间，O(状态数) 额外空间。扫描文本时 u=ac.a[u].go[c]，len[u] 就是当前位置结尾的最短匹配长度；文本仍须为小写字母。辅助函数不修改自动机，也不影响 count(text)，可对同一自动机的不同登记集合重复调用。

```
vector<int> ac_shortest(const AhoCorasick &ac, const vector<pair<int, int>> &ends)
{
    assert(ac.built);
    vector<int> len(ac.a.size(), INT_MAX);
    for (auto [u, n] : ends)
    {
        assert(0 < u && u < (int)len.size() && 0 < n && n < INT_MAX);
        len[u] = min(len[u], n);
    }
    for (int u : ac.order) len[u] = min(len[u], len[ac.a[u].fail]);
    return len;
}
```

应用题补充：AC 最短后缀匹配（本地接口示例）

题意：输入模式数、各个非空小写模式串及文本，逐位置输出以该位置结尾的最短模式长度，无匹配输出 0。先登记终止节点与长度，再 build 和传播。示例对应附件 min_len 变体，只作本地接口演示，不是 OJ 提交记录。

所需模板：AhoCorasick（第 16 页，4.1 节）、ac_shortest（第 22 页，4.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://oi-wiki.org/string/ac-automaton/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    AhoCorasick ac;
    vector<pair<int, int>> ends;
    while (n--)
    {
        string s;
        cin >> s;
        ends.push_back({ac.add(s), (int)s.size()});
    }
    ac.build();
    auto len = ac_shortest(ac, ends);
    string s;
    cin >> s;
    int u = 0;
    for (char c : s)
    {
        u = ac.a[u].go[c - 'a'];
        cout << (len[u] == INT_MAX ? 0 : len[u]) << ' ';
    }
    cout << '\n';
}
```

4.3 AC 自动机带权匹配

代码依赖：AhoCorasick（第 16 页，4.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

构造接收 (小写模式串, long long 权重) 列表；重复串权重相加，可负、可零。sum[u] 为当前状态所有后缀模式的权重和，query(text) 累加全部重叠匹配，O(|text|)。允许空串，在每个边界匹配，共 |text|+1 次；空模式集合返回 0。构造 O(26V+L+n)，空间 O(26V+n)，L 为总模式长。对象自持有 ac 和 sum，可复制、移动；构造后成员只读，重置赋新对象。所有中间和及答案须在 long long 内。

```
struct ACWeighted
{
    AhoCorasick ac;
    vector<long long> sum;

    ACWeighted(const vector<pair<string, long long>> &patterns = {})
    {
        vector<pair<int, long long>> ends;
        for (const auto &[s, w] : patterns)
        {
            int u = s.empty() ? 0 : ac.add(s);
            ends.push_back({u, w});
        }
        ac.build();
        sum.resize(ac.a.size());
        for (auto [u, w] : ends) sum[u] += w;
        for (int u : ac.order) sum[u] += sum[ac.a[u].fail];
    }

    long long query(const string &s) const
    {
        long long ans = sum[0];
        int u = 0;
        for (char c : s)
        {
            assert('a' <= c && c <= 'z');
            u = ac.a[u].go[c - 'a'];
            ans += sum[u];
        }
        return ans;
    }
};
```

接口用法演示：静态带权多模式查询（接口演示）

演示约定：输入 n,q，随后 n 组带引号模式串及有符号权重，再读 q 个带引号文本。双引号允许表示空串。输出全部重叠匹配的权重和；空模式在每个边界匹配。

所需模板：AhoCorasick（第 16 页，4.1 节）、ACWeighted（第 24 页，4.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/string/ac-automaton/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<pair<string, long long>> patterns(n);
    for (auto &[s, w] : patterns) cin >> quoted(s) >> w;
    ACWeighted ac(patterns);
    while (q--)
    {
        string s;
        cin >> quoted(s);
        cout << ac.query(s) << '\n';
    }
}
```

4.4 动态 AC：二进制分组重建

代码依赖：ACWeighted（第 24 页，4.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

add(s,w) 累加模式权重；插入计数用 +1，删除用-1，合法删除由调用者保证。query(s) 求所有模式出现次数乘当前权重之和，支持重复、负权和空串，边界同 ACWeighted。第 k 组为空或含 2^k 次更新；进位合并后重建。M 次更新、总长度 L，累计构建 O(26(L+M)log(M+2))，查询 O((|s|+1)log(M+2))，空间 O(26(L+M))；一次更新可线性重建，非最坏对数。历史删除不会回收更新记录，空间按全部历史更新计。block/ac 只读；重置赋新对象。所有组内及跨组中间和须在 long long 内。

```
struct DynamicAC
{
    vector<vector<pair<string, long long>>> block;
    vector<ACWeighted> ac;

    void add(string s, long long w)
    {
        vector<pair<string, long long>> cur;
        cur.push_back({move(s), w});
        int k = 0;
        while (k < (int)block.size() && !block[k].empty())
        {
            for (auto &p : block[k]) cur.push_back(move(p));
            block[k].clear();
            ac[k] = ACWeighted();
            k++;
        }
        if (k == (int)block.size())
        {
            block.emplace_back();
            ac.emplace_back();
        }
        block[k] = move(cur);
        ac[k] = ACWeighted(block[k]);
    }

    long long query(const string &s) const
    {
        long long ans = 0;
        for (int k = 0; k < (int)block.size(); k++)
        {
            if (!block[k].empty()) ans += ac[k].query(s);
        }
        return ans;
    }
};
```

应用题补充：Codeforces 710F / String Set Queries

题意：在线维护字符串集合。1 插入，2 删除，3 询问文本中所有集合模式的总出现次数。题目保证新增串从未加入、删除串存在；每次回答必须 endl 刷新后再读下一条，不能预读全部操作。用 +1/-1 更新权重。

所需模板：AhoCorasick（第 16 页，4.1 节）、ACWeighted（第 24 页，4.3 节）、DynamicAC（第 26 页，4.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/problemset/problem/710/F

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m;
    cin >> m;
    DynamicAC dict;
    while (m--)
    {
        int t;
        string s;
        cin >> t >> s;
        if (t == 3)
        {
            cout << dict.query(s) << endl;
        }
        else
        {
            dict.add(move(s), t == 1 ? 1 : -1);
        }
    }
}
```

第五章　后缀数组 (SA)

5.1　后缀数组简介

5.1.1　线性后缀数组（SA-IS）

SAIS(s,alphabet)：整数符号 0..alphabet-1，1≤alphabet<INT_MAX，长度 n<INT_MAX；SAIS(string) 按 unsigned byte 处理，支持零字节。sa/rk/lcp 均长 n，0-based；lcp[0]=0，其余为相邻后缀 LCP。无需哨兵，空串结果为空。诱导排序时间/空间 O(n+alphabet)，递归深度 O(log n)；任意值离散化另需 O(n log n)。保留倍增 SuffixArray；SuffixLCP 不直接接受本类型。

```
struct SAIS
{
    vector<int> sa, rk, lcp;

    SAIS(const string &s)
    {
        assert(s.size() < INT_MAX);
        vector<int> a;
        a.reserve(s.size());
        for (unsigned char c : s) a.push_back(c);
        *this = SAIS(a, 256);
    }

    SAIS(const vector<int> &s, int alphabet)
    {
        assert(s.size() < INT_MAX && 1 <= alphabet && alphabet < INT_MAX);
        for (int c : s) assert(0 <= c && c < alphabet);
        int n = s.size();
        sa = sort_suffixes(s, alphabet);
        rk.resize(n);
        lcp.assign(n, 0);
        for (int i = 0; i < n; i++) rk[sa[i]] = i;
        for (int i = 0, k = 0; i < n; i++)
        {
            if (!rk[i])
            {
                k = 0;
                continue;
            }
            int j = sa[rk[i] - 1];
            while (k < n - i && k < n - j && s[i + k] == s[j + k]) k++;
            lcp[rk[i]] = k;
            if (k) k--;
        }
    }
```

类型与桶内诱导（接上页同一结构体）：

```
private:
    static vector<int> sort_suffixes(const vector<int> &s, int alphabet)
    {
        int n = s.size();
        if (!n) return {};
        if (n == 1) return {0};
        vector<int> sa(n), lo(alphabet + 1), hi(alphabet + 1);
        vector<bool> type(n);
        for (int i = n - 2; i >= 0; i--)
            type[i] = s[i] == s[i + 1] ? type[i + 1] : s[i] < s[i + 1];
        for (int i = 0; i < n; i++)
            if (type[i]) lo[s[i] + 1]++;
            else hi[s[i]]++;
        for (int c = 0; c < alphabet; c++)
        {
            hi[c] += lo[c];
            lo[c + 1] += hi[c];
        }
        auto induce = [&](const vector<int> &lms)
        {
            fill(sa.begin(), sa.end(), -1);
            vector<int> pos = hi;
            for (int i : lms) sa[pos[s[i]]++] = i;
            pos = lo;
            sa[pos[s[n - 1]]++] = n - 1;
            for (int i = 0; i < n; i++)
            {
                int v = sa[i];
                if (v > 0 && !type[v - 1]) sa[pos[s[v - 1]]++] = v - 1;
            }
            pos = lo;
            for (int i = n - 1; i >= 0; i--)
            {
                int v = sa[i];
                if (v > 0 && type[v - 1]) sa[--pos[s[v - 1] + 1]] = v - 1;
            }
        };
```

LMS 命名与递归（接上页同一结构体）：

```
        vector<int> id(n, -1), lms;
        for (int i = 1; i < n; i++)
            if (!type[i - 1] && type[i])
            {
                id[i] = lms.size();
                lms.push_back(i);
            }
        induce(lms);
        int m = lms.size();
        if (m)
        {
            vector<int> order, reduced(m);
            for (int v : sa)
                if (id[v] >= 0) order.push_back(v);
            int names = 1;
            for (int i = 1; i < m; i++)
            {
                int a = order[i - 1], b = order[i];
                int x = id[a] + 1 < m ? lms[id[a] + 1] : n;
                int y = id[b] + 1 < m ? lms[id[b] + 1] : n;
                bool same = x - a == y - b;
                if (same)
                {
                    while (a < x && s[a] == s[b]) a++, b++;
                    same = a < n && b < n && s[a] == s[b];
                }
                if (!same) names++;
                reduced[id[order[i]]] = names - 1;
            }
            order = sort_suffixes(reduced, names);
            for (int &i : order) i = lms[i];
            induce(order);
        }
        return sa;
    }
};
```

模板题使用：Library Checker suffixarray

题意：输入长度 1..500000 的小写字符串，按字典序输出所有非空后缀的 0-based 起点。独立 SA-IS 实现；无需附加哨兵，不继承倍增版本的线上记录。

所需模板：SAIS（第 28 页，5.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/suffixarray

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    SAIS suffix(s);
    for (int i = 0; i < (int)s.size(); i++)
        cout << suffix.sa[i] << (i + 1 == (int)s.size() ? '\n' : ' ');
}
```

5.1.2 倍增后缀数组与 LCP

构造 SuffixArray(s) 支持任意字节串，按 unsigned char 的 0..255 数值比较，包含内嵌零字节；也可构造 SuffixArray(a,alphabet)，整数序列要求 0≤a[i]<alphabet、1≤alphabet<INT_MAX。重新赋值 SuffixArray(a,alphabet) 可重建。长度须小于 INT_MAX。sa[k] 为第 k 个非空后缀起点，rk[i] 为排名，全部 0-based；lcp[k] 为 sa[k-1] 与 sa[k] 的最长公共前缀，lcp[0]=0。空序列合法，三数组均为空。无需手动追加哨兵，也不输出空后缀。每轮按第二关键字已有顺序、第一关键字稳定计数排序，倍增 O(n log n+alphabet)，空间 O(n+alphabet)；字节串 alphabet=256。大整数先自行保序离散化，避免按最大原值开桶。Kasai 求 lcp 为 O(n)。不同起点 x,y 的 LCP 为 lcp[min(rk[x],rk[y])+1..max(rk[x],rk[y])] 的最小值；本结构不内建 RMQ，需要频繁查询时另接静态 RMQ。同一起点的 LCP 为 n-x。覆盖倍增构造，不替代 DC3/SA-IS。

```
struct SuffixArray
{
    vector<int> sa, rk, lcp;

    SuffixArray(const string &s)
    {
        vector<int> a;
        a.reserve(s.size());
        for (unsigned char c : s) a.push_back(c);
        *this = SuffixArray(a, 256);
    }

    SuffixArray(const vector<int> &s, int alphabet)
    {
        assert(s.size() < INT_MAX);
        assert(1 <= alphabet && alphabet < INT_MAX);
        int n = (int)s.size();
        sa.resize(n);
        rk.resize(n);
        lcp.assign(n, 0);
        if (!n) return;
        vector<int> order(n), old(n), cnt(max(n + 1, alphabet + 1));
        int m = alphabet;
        for (int i = 0; i < n; i++)
        {
            assert(0 <= s[i] && s[i] < alphabet);
            rk[i] = s[i] + 1;
            cnt[rk[i]]++;
        }
        for (int i = 1; i <= m; i++) cnt[i] += cnt[i - 1];
        for (int i = n - 1; i >= 0; i--) sa[--cnt[rk[i]]] = i;
```

```
        for (int k = 1; k < n; k *= 2)
        {
            int p = 0;
            for (int i = n - k; i < n; i++) order[p++] = i;
            for (int i : sa)
                if (i >= k) order[p++] = i - k;
            fill(cnt.begin(), cnt.begin() + m + 1, 0);
            for (int x : rk) cnt[x]++;
            for (int i = 1; i <= m; i++) cnt[i] += cnt[i - 1];
            for (int i = n - 1; i >= 0; i--) sa[--cnt[rk[order[i]]]] = order[i];
            old.swap(rk);
            auto second = [&](int i)
            {
                return i < n - k ? old[i + k] : 0;
            };
            m = 1;
            rk[sa[0]] = m;
            for (int i = 1; i < n; i++)
            {
                int a = sa[i - 1], b = sa[i];
                if (old[a] != old[b] || second(a) != second(b)) m++;
                rk[b] = m;
            }
            if (m == n) break;
        }
        for (int i = 0; i < n; i++) rk[sa[i]] = i;
        for (int i = 0, k = 0; i < n; i++)
        {
            if (!rk[i])
            {
                k = 0;
                continue;
            }
            int j = sa[rk[i] - 1];
            while (k < n - i && k < n - j && s[i + k] == s[j + k]) ++k;
            lcp[rk[i]] = k;
            if (k) --k;
        }
    }
};
```

模板题使用：Luogu P3809

题意：把所有非空后缀按字典序排序，输出各后缀在原串中的起点。模板 sa 为 0-based，题目输出为 1-based，因此加 1。

所需模板：SuffixArray（第 32 页，5.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3809

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::string s;
    std::cin >> s;
    SuffixArray suffix(s);
    for (int i = 0; i < (int)s.size(); i++)
        std::cout << suffix.sa[i] + 1 << (i + 1 == (int)s.size() ? '\n' : ' ');
}
```

模板题使用：Library Checker longest_common_substring

题意：求两个小写字符串的一段最长公共子串，输出两串各自 0-based 半开区间，各串长度至 50 万。整数拼接以 26 为分隔符、alphabet=27；扫描异来源相邻后缀，并将 LCP 截断至原串边界。

所需模板：SuffixArray（第 32 页，5.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/longest_common_substring

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s, t;
    cin >> s >> t;
    int n = s.size(), m = t.size();
    vector<int> a;
    for (char c : s) a.push_back(c - 'a');
    a.push_back(26);
    for (char c : t) a.push_back(c - 'a');
    SuffixArray suffix(a, 27);
    int best = 0, left = 0, right = 0;
    for (int k = 1; k < (int)a.size(); k++)
    {
        int x = suffix.sa[k - 1], y = suffix.sa[k];
        if (x > y) swap(x, y);
        if (x >= n || y <= n) continue;
        int len = min({suffix.lcp[k], n - x, n + 1 + m - y});
        if (len > best)
        {
            best = len;
            left = x;
            right = y - n - 1;
        }
    }
    cout << left << ' ' << left + best << ' ' << right << ' ' << right + best << '\n';
}
```

5.1.3 后缀 LCP 与子串比较

代码依赖：SuffixArray（第 32 页，5.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

先构造 SuffixArray，再构造 SuffixLCP(sa)。重新赋值 SuffixLCP(sa) 可建新表。复制所需排名与 LCP 数据，不保存原字符串或源对象引用，源对象可销毁或重建。query(x,y) 返回 s[x..n) 与 s[y..n) 的最长公共前缀，$0 \le x,y \le n$，允许空后缀；同起点返回 n-x。compare(l1,r1,l2,r2) 比较两个 0-based 半开子串，允许空串，返回 -1/0/1。若公共前缀覆盖较短子串，按长度比较，否则原后缀排名给出首个不同字符的顺序。字节排序及整数字母表约束继承 SuffixArray。预处理稀疏表 O(n log n) 时间、空间，每次查询 O(1)，无需哈希和原串。n、rk、lg、st 可读，禁止调用方破坏。

```
struct SuffixLCP
{
    int n;
    vector<int> rk, lg;
    vector<vector<int>> st;

    SuffixLCP(const SuffixArray &suffix)
    {
        n = suffix.sa.size();
        rk = suffix.rk;
        lg.assign(n + 1, 0);
        for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;
        st.assign(lg[n] + 1, {});
        st[0] = suffix.lcp;
        for (int k = 1; k <= lg[n]; k++)
        {
            int len = 1 << k;
            st[k].resize(n - len + 1);
            for (int i = 0; i <= n - len; i++)
                st[k][i] = min(st[k - 1][i], st[k - 1][i + len / 2]);
        }
    }
```

```
    int query(int x, int y) const
    {
        assert(0 <= x && x <= n && 0 <= y && y <= n);
        if (x == y || x == n || y == n) return n - max(x, y);
        int a = rk[x], b = rk[y];
        if (a > b) swap(a, b);
        int k = lg[b - a];
        return min(st[k][a + 1], st[k][b - (1 << k) + 1]);
    }

    // Compare [l1,r1) and [l2,r2); return -1, 0 or 1.
    int compare(int l1, int r1, int l2, int r2) const
    {
        assert(0 <= l1 && l1 <= r1 && r1 <= n);
        assert(0 <= l2 && l2 <= r2 && r2 <= n);
        int a = r1 - l1, b = r2 - l2;
        if (query(l1, l2) >= min(a, b)) return (a > b) - (a < b);
        return rk[l1] < rk[l2] ? -1 : 1;
    }
};
```

接口用法演示：静态后缀查询（自定义协议）

演示约定：自定义接口演示，非官方题目或 OJ 驱动。首行 n q，次行恰为 n 个小写字母（n=0 时保留空行）；0≤n,q≤200000 仅为演示上限。操作 0 x y 输出 s[x..n)、s[y..n) 的 LCP；1 l1 r1 l2 r2 比较两个半开子串，输出-1/0/1；2 x y 输出前缀 s[0..x)、s[0..y) 的最长公共后缀长度，LCS 在此不是子序列。所有端点 0..n，允许空区间；同一起点 LCP 为 n-x，同一前缀 LCS 为 x。反向结构必须从原串倒序构造。子串 LCP 取 min(query(l1,l2),r1-l1,r2-l2)，子串公共后缀取 min(prefix_lcs(reversed,r1,r2),r1-l1,r2-l2)。预处理 O(n log n) 时间/空间，各查询 O(1)；字符串修改后重建。

所需模板：SuffixArray（第 32 页，5.1.2 节）、SuffixLCP（第 35 页，5.1.3 节）、prefix_lcs（第 38 页，5.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/src/compact/suffix_lcp.hpp

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    assert(0 <= n && n <= 200000 && 0 <= q && q <= 200000);
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    string s;
    getline(cin, s); // Keep the empty line when n = 0; do not use cin >> ws.
    assert((int)s.size() == n);
    for (char c : s) assert('a' <= c && c <= 'z');
    SuffixLCP forward{SuffixArray(s)};
    string r(s.rbegin(), s.rend());
    SuffixLCP reversed{SuffixArray(r)};
    while (q--)
    {
        int op;
        cin >> op;
        if (op == 0)
        {
            int x, y;
            cin >> x >> y;
            cout << forward.query(x, y) << '\n';
        }
        else if (op == 1)
        {
            int l1, r1, l2, r2;
            cin >> l1 >> r1 >> l2 >> r2;
            cout << forward.compare(l1, r1, l2, r2) << '\n';
        }
        else
        {
            assert(op == 2);
            int x, y;
            cin >> x >> y;
            cout << prefix_lcs(reversed, x, y) << '\n';
        }
    }
}
```

接口用法演示：kuangbin DA 来源模型：反串与 LCP 求最长回文

演示约定：来源程序协议：读入非空字符串直到 EOF，每组输出最靠左的最长回文子串。组合原串、独立分隔符与反串，通过奇偶中心的 LCP 求半径；i=0 的偶中心查询空后缀，返回 0。新实现按 unsigned char 加 2，分隔符为 1；长度须 ≤(INT_MAX−2)/2 且满足内存限制。O(n log n) 时间空间。此为来源应用协议演示，不是独立线上模板题或 AC；仅需最长回文时优先用 Manacher。

所需模板：SuffixArray（第 32 页，5.1.2 节）、SuffixLCP（第 35 页，5.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/kuangbin/ACM-ICPC

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    while (cin >> s)
    {
        assert(s.size() <= (INT_MAX - 2) / 2);
        int len = s.size();
        int n = 2 * len + 1;
        vector<int> a(n);
        for (int i = 0; i < len; i++)
        {
            a[i] = (unsigned char)s[i] + 2;
            a[len + 1 + i] = (unsigned char)s[len - 1 - i] + 2;
        }
        a[len] = 1;
        SuffixArray suffix(a, 258);
        SuffixLCP lcp(suffix);
        int best = 0, start = 0;
        for (int i = 0; i < len; i++)
        {
            int radius = lcp.query(i, n - i);
            if (2 * radius > best)
            {
                best = 2 * radius;
                start = i - radius;
            }
            radius = lcp.query(i, n - i - 1);
            if (2 * radius - 1 > best)
            {
                best = 2 * radius - 1;
                start = i - radius + 1;
            }
        }
        cout << s.substr(start, best) << '\n';
    }
    return 0;
}
```

5.1.4 两个前缀的最长公共后缀

代码依赖：SuffixLCP（第 35 页，5.1.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

prefix_lcs(reversed,x,y) 返回原序列 s[0..x) 与 s[0..y) 的最长公共后缀，端点满足 0≤x,y≤n。reversed 必须在原序列的反转序列上构造 SuffixLCP，不能传正向对象；转换为 reversed.query(n-x,n-y)，单次 O(1)。只需前向 LCP 时不必构造反向对象；该接口中的 LCS 是最长公共后缀，不是最长公共子序列。对于两个任意子串，以右端点查询后还需截断到两个子串长度的最小值。

```
// reversed must be built on the reversed original sequence.
int prefix_lcs(const SuffixLCP &reversed, int x, int y)
{
    assert(0 <= x && x <= reversed.n && 0 <= y && y <= reversed.n);
    return reversed.query(reversed.n - x, reversed.n - y);
}
```

联合使用示例见第 36 页（静态后缀查询（自定义协议）），其中直接调用本模板。

5.1.5 重复子串 AA 的起止位置计数

代码依赖：SuffixLCP（第 35 页，5.1.3 节）、prefix_lcs（第 38 页，5.1.4 节）。抄写时一并准备；标准库类型不列入算法依赖。

square_counts(forward,reversed) 返回 (start,finish) 两个长度 n 的 long long 数组，start[i]/finish[i] 为开始/结束于位置 i 的非空 AA 型子串个数；结束位置是最后一个字符下标，均 0-based。两对象必须分别对应同一原序列及其反序列，长度相同，允许空序列。枚举半长 len，并以其整数倍 i 为锚点；向左匹配截断为 len-1，向右截断为 len，由 LCP/LCS 得出合法起点连续区间，差分累计。左侧不能放宽到 len，否则相邻锚点会重复计数。预处理查询器后，此函数 O(n log n) 时间、O(n) 额外空间。不同出现位置、不同半长分别计数；不要求 A 为最小周期。P1117 的 AABB 拆分数为所有分界点 finish[i]*start[i+1] 之和，累加类型需按题目范围选择，函数本身只返回起止计数。

```
// Count nonempty squares AA by their starting and ending positions.
pair<vector<long long>, vector<long long>> square_counts(const SuffixLCP &forward,
                                                         const SuffixLCP &reversed)
{
    int n = forward.n;
    assert(reversed.n == n);
    vector<long long> start(n + 1), finish(n + 1);
    for (int len = 1; len <= n / 2; len++)
        for (int i = 0; i < n - len; i += len)
        {
            int j = i + len;
            int left = min(len - 1, prefix_lcs(reversed, i, j));
            int right = min(len, forward.query(i, j));
            if (left + right < len) continue;
            int l = i - left, r = i + right - len;
            start[l]++;
            start[r + 1]--;
            finish[l + 2 * len - 1]++;
            finish[r + 2 * len]--;
        }
```

```
    for (int i = 1; i < n; i++)
    {
        start[i] += start[i - 1];
        finish[i] += finish[i - 1];
    }
    start.resize(n);
    finish.resize(n);
    return {start, finish};
}
```

应用题补充：Luogu P1117 [NOI2016] 优秀的拆分

题意：统计所有子串的非空 AABB 拆分，不同出现位置、不同 A/B 长度分别计数，允许 A=B。T 为 1 至 10，每组小写串长度不超过 30000。正向与反向 SuffixLCP 必须来自同一字符串；start[i] 和 finish[i] 分别计开始/结束于字符 i 的 AA。每个分界点贡献 finish[i]*start[i+1]，以 long long 累加。此为 NOI 比赛应用题，尚未计入独立模板题覆盖。

所需模板：SuffixArray（第 32 页，5.1.2 节）、SuffixLCP（第 35 页，5.1.3 节）、prefix_lcs（第 38 页，5.1.4 节）、square_counts（第 38 页，5.1.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1117

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int tests;
    std::cin >> tests;
    while (tests--)
    {
        std::string s;
        std::cin >> s;
        int n = s.size();
        SuffixLCP forward{SuffixArray(s)};
        std::reverse(s.begin(), s.end());
        SuffixLCP backward{SuffixArray(s)};
        auto [start, finish] = square_counts(forward, backward);
        long long answer = 0;
        for (int i = 0; i + 1 < n; i++) answer += finish[i] * start[i + 1];
        std::cout << answer << '\n';
    }
}
```

5.1.6 后缀数组：DC3 线性构造

DC3(string) 按 unsigned char 比较全部字节（含 0/255）；DC3(vector<int>,alphabet) 要求每项在 [0,alphabet)，1≤alphabet<INT_MAX−3，长度 n<INT_MAX−3。空输入合法。sa 为所有非空后缀的 0-based 起点，rk[sa[i]]=i；lcp[0]=0，lcp[i] 为 sa[i−1] 与 sa[i] 的公共前缀长度。三数组均长 n，空输入均为空，不含空后缀；输入不被改写，无需调用者补零或三倍数组。内部把符号加一，0 只作哨兵；n mod3=1 时补一个虚拟采样点，归并时跳过。稳定基数排序三元组、递归命名、排序余 0 类并合并；时间空间均 O(n+alphabet)，仅 alphabet=O(n) 或固定字节集时称线性。稀疏大整数先保序离散化，其比较排序代价另计。递归深度 O(log(n+1))，私有 sort_suffixes 要求正符号，用户只用构造及结果数组。与 SuffixArray/SAIS 接口约定一致，无线上性能优越性声明。

```
struct DC3
{
    vector<int> sa, rk, lcp;

    DC3(const string &s)
    {
        vector<int> a;
        for (unsigned char c : s)
            a.push_back(c);
        *this = DC3(a, 256);
    }

    DC3(const vector<int> &s, int alphabet)
    {
        assert(s.size() < INT_MAX - 3 && 1 <= alphabet && alphabet < INT_MAX - 3);
        int n = s.size();
        vector<int> a(n);
        for (int i = 0; i < n; i++)
        {
            assert(0 <= s[i] && s[i] < alphabet);
            a[i] = s[i] + 1;
        }
        sa = sort_suffixes(move(a), alphabet);
        rk.resize(n);
        lcp.assign(n, 0);
        for (int i = 0; i < n; i++)
            rk[sa[i]] = i;
        for (int i = 0, k = 0; i < n; i++)
        {
            if (!rk[i])
            {
                k = 0;
                continue;
            }
            int j = sa[rk[i] - 1];
            while (k < n - i && k < n - j && s[i + k] == s[j + k])
                k++;
            lcp[rk[i]] = k;
            if (k)
                k--;
        }
    }
```

采样三元组排序与递归命名（接上页同一结构体）：

```
private:
    static vector<int> sort_suffixes(vector<int> s, int alphabet)
    {
        int n = s.size();
        if (!n)
            return {};
        if (n == 1)
            return {0};
        s.resize(n + 3);
        int n0 = (n + 2) / 3;
        int extra = n % 3 == 1;
        vector<int> sample, zero;
        for (int i = 0; i < n + extra; i++)
            if (i % 3)
                sample.push_back(i);
        auto radix = [&](vector<int> &a, int shift)
        {
            vector<int> count(alphabet + 1), b(a.size());
            for (int i : a)
                count[s[i + shift]]++;
            for (int i = 1; i <= alphabet; i++)
                count[i] += count[i - 1];
            for (int i = (int)a.size() - 1; i >= 0; i--)
                b[--count[s[a[i] + shift]]] = a[i];
            a.swap(b);
        };
        for (int shift = 2; shift >= 0; shift--)
            radix(sample, shift);
        int names = 0;
        vector<int> reduced(sample.size()), order(sample.size());
        array<int, 3> last{-1, -1, -1};
        for (int i : sample)
        {
            array<int, 3> key{s[i], s[i + 1], s[i + 2]};
            if (key != last)
                names++;
            last = key;
            reduced[i / 3 + (i % 3 == 2 ? n0 : 0)] = names;
        }
        if (names < (int)sample.size())
            order = sort_suffixes(move(reduced), names);
        else
            for (int i = 0; i < (int)reduced.size(); i++)
                order[reduced[i] - 1] = i;
```

恢复采样排名并合并（接上页同一结构体）：

```
        vector<int> rank(n + 3);
        for (int k = 0; k < (int)order.size(); k++)
        {
            int i = order[k];
            int p = i < n0 ? 3 * i + 1 : 3 * (i - n0) + 2;
            sample[k] = p;
            rank[p] = k + 1;
            if (p % 3 == 1)
                zero.push_back(p - 1);
        }
        radix(zero, 0);
        vector<int> result;
        int a = extra, b = 0;
        while (a < (int)sample.size() && b < (int)zero.size())
        {
            int i = sample[a], j = zero[b];
            bool first;
            if (i % 3 == 1)
                first = pair{s[i], rank[i + 1]} < pair{s[j], rank[j + 1]};
            else
                first = tuple{s[i], s[i + 1], rank[i + 2]}
                      < tuple{s[j], s[j + 1], rank[j + 2]};
            if (first)
                result.push_back(sample[a++]);
            else
                result.push_back(zero[b++]);
        }
        while (a < (int)sample.size())
            result.push_back(sample[a++]);
        while (b < (int)zero.size())
            result.push_back(zero[b++]);
        return result;
    }
};
```

模板题使用：Library Checker suffixarray（DC3）

题意：输入长度 1..500000 的小写字符串，输出所有非空后缀按字典序排列的 0-based 起点。DC3 独立程序，无需附加哨兵；原倍增或 SA-IS 用法的线上记录不转移到本程序。

所需模板：DC3（第 40 页，5.1.6 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/suffixarray

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    DC3 d(s);
    for (int i = 0; i < (int)d.sa.size(); i++)
        cout << d.sa[i] << (i + 1 == (int)d.sa.size() ? '\n' : ' ');
    return 0;
}
```

接口用法演示：稠密整数后缀数组、逆排名及 LCP

演示约定：输入 n,alphabet 与 n 个 [0,alphabet) 整数，分别输出 sa、rk、lcp 三行。允许 n=0，此时仍输出三个空行。lcp[0]=0，其余对当前排名与前一名后缀比较；输入符号 0 可用，不应手工插入空后缀哨兵。API 演示，不是线上题目或 AC。

所需模板：DC3（第 40 页，5.1.6 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/string/sa/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, alphabet;
    cin >> n >> alphabet;
    vector<int> s(n);
    for (int &c : s)
        cin >> c;
    DC3 d(s, alphabet);
    for (int x : d.sa)
        cout << x << ' ';
    cout << '\n';
    for (int x : d.rk)
        cout << x << ' ';
    cout << '\n';
    for (int x : d.lcp)
        cout << x << ' ';
    cout << '\n';
    return 0;
}
```

第六章　后缀自动机 (SAM)

6.1　后缀自动机

extend(c) 接收整数 0..25，每次追加一个小写字符；last 为整串状态，a[0] 为根，根 link=-1。a[u].len 为状态最大长度，非根代表 (len[link[u]],len[u]] 且具有同一 endpos 集合。occ 是原始种子：真实扩展为 1、克隆为 0，不能直接当出现次数。counts() 按长度倒序汇总后返回副本，不改结构，重复查询及查询后继续追加安全；根计数为 n，非空子串只看 u>0。distinct() 为不同非空子串数。重赋值 SuffixAutomaton() 清空；固定 26 字母表构建 O(n)，counts/distinct 为 O(n)，空间 O(26n)。如需前缀位置对应状态，在每次 extend 后保存 last；本结构不输出广义 SAM 或最小 DFA 状态数。

```
struct SuffixAutomaton
{
    struct Node
    {
        array<int, 26> go{};
        int link = -1, len = 0;
        long long occ = 0;
    };

    vector<Node> a{Node{}};
    int last = 0;
```

接上页同一结构体：

```
    void extend(int c)
    {
        assert(0 <= c && c < 26);
        int cur = (int)a.size();
        a.push_back(Node{});
        a[cur].len = a[last].len + 1;
        a[cur].occ = 1;
        int p = last;
        while (p != -1 && !a[p].go[c])
        {
            a[p].go[c] = cur;
            p = a[p].link;
        }
        if (p == -1)
            a[cur].link = 0;
        else
        {
            int q = a[p].go[c];
            if (a[p].len + 1 == a[q].len)
                a[cur].link = q;
            else
            {
                int clone = (int)a.size();
                Node copy = a[q];
                a.push_back(copy);
                a[clone].len = a[p].len + 1;
                a[clone].occ = 0;
                while (p != -1 && a[p].go[c] == q)
                {
                    a[p].go[c] = clone;
                    p = a[p].link;
                }
                a[q].link = a[cur].link = clone;
            }
        }
        last = cur;
    }
```

接上页同一结构体：

```
    vector<long long> counts() const
    {
        vector<int> bucket(a[last].len + 1), order(a.size());
        vector<long long> ans(a.size());
        for (auto &v : a) ++bucket[v.len];
        partial_sum(bucket.begin(), bucket.end(), bucket.begin());
        for (int i = 0; i < (int)a.size(); i++)
        {
            order[--bucket[a[i].len]] = i;
            ans[i] = a[i].occ;
        }
        for (int i = (int)order.size() - 1; i > 0; i--)
            ans[a[order[i]].link] += ans[order[i]];
        return ans;
    }

    long long distinct() const
    {
        long long ans = 0;
        for (int i = 1; i < (int)a.size(); i++) ans += a[i].len - a[a[i].link].len;
        return ans;
    }
};
```

模板题使用：Library Checker number_of_substrings

题意：给小写字母串，求不同非空子串的个数。extend 接收 0..25 的整数，故字符减去 a；所有字符追加后调用 distinct，结果使用 long long。

所需模板：SuffixAutomaton（第 45 页，6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/number_of_substrings

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    SuffixAutomaton sam;
    for (char c : s) sam.extend(c - 'a');
    cout << sam.distinct() << '\n';
}
```

模板题使用：Luogu P3804

题意：长度至 100 万的小写串，只考虑出现次数大于 1 的子串，最大化长度乘次数。counts 按长度汇总，取非根状态 len*count；无重复子串输出 0，乘积使用 long long。

所需模板：SuffixAutomaton（第 45 页，6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3804

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    SuffixAutomaton sam;
    for (char c : s) sam.extend(c - 'a');
    auto count = sam.counts();
    long long answer = 0;
    for (int u = 1; u < (int)sam.a.size(); u++)
    {
        if (count[u] > 1) answer = max(answer, count[u] * sam.a[u].len);
    }
    cout << answer << '\n';
}
```

接口用法演示：区间不同子串（kuangbin HDU4622 模型）

演示约定：来源协议：T 组，每组非空小写字母串 s、q 及 q 个 1-based 闭区间 [l,r]，输出该区间内不同非空子串数量。每个左端点新建 SAM，每次 extend 后累加新 last 的 len 减 link.len；克隆只重分配旧子串，不另增贡献。固定 26 字母表，预处理 $O(n^2)$，查询 O(1)，答案表 $O(n^2)$、临时 SAM O(n) 空间。不要在内层循环调用扫描所有状态的 distinct()，也不能跨左端点复用旧 SAM。当前为来源模型/API 用法，官方题面、资源限制及线上 AC 未核验。

所需模板：SuffixAutomaton（第 45 页，6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://acm.hdu.edu.cn/showproblem.php?pid=4622

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        string s;
        cin >> s;
        int n = (int)s.size();
        vector<vector<long long>> ans(n, vector<long long>(n));
        for (int l = 0; l < n; l++)
        {
            SuffixAutomaton sam;
            long long sum = 0;
            for (int r = l; r < n; r++)
            {
                sam.extend(s[r] - 'a');
                int u = sam.last;
                sum += sam.a[u].len - sam.a[sam.a[u].link].len;
                ans[l][r] = sum;
            }
        }
        int q;
        cin >> q;
        while (q--)
        {
            int l, r;
            cin >> l >> r;
            cout << ans[l - 1][r - 1] << '\n';
        }
    }
}
```

6.2 SAM 字典序子串查询

SAMLex(sam,w={}) 引用已建好的 SuffixAutomaton 或已 build 的 GeneralSAM；索引使用期间 sam 必须存活且保持不变。默认每个非根状态权重 1，得到不同子串；传单串 sam.counts() 按出现次数计数。自定义 w 长度为状态数、非负 long long，每条到达该状态的串有 w[u] 份，根权重强制 0。sum[u] 为本状态及后续路径权重和，超过 LLONG_MAX 截断，仍正确支持所有 1≤k≤LLONG_MAX。kth(k,alphabet) 返回 optional<string>，不足或 k≤0 为 nullopt；默认 a..z，alphabet 必须是 26 字母的排列。短串先于其扩展，支持重复查询与自定义字母序。构建 O(26V+L)，空间 O(V+L)，L 为最长串长；单次查询 O(26× 答案长度)，额外空间为答案。weight/sum 只读，不支持扩展 SAM 后沿用旧索引。

```
// Keep sam alive and unchanged while this index is used.
template <class S>
struct SAMLex
{
    const S &sam;
    vector<long long> weight, sum;

    SAMLex(const S &sam, vector<long long> w = {}) : sam(sam), weight(move(w))
    {
        int n = sam.a.size(), length = 0;
        if (weight.empty()) weight.assign(n, 1);
        assert((int)weight.size() == n);
        weight[0] = 0;
        sum = weight;
        for (auto &v : sam.a) length = max(length, v.len);
        vector<int> bucket(length + 1), order(n);
        for (auto &v : sam.a) bucket[v.len]++;
        for (int i = 1; i <= length; i++) bucket[i] += bucket[i - 1];
        for (int i = 0; i < n; i++) order[--bucket[sam.a[i].len]] = i;
        for (int i = n - 1; i >= 0; i--)
        {
            int u = order[i];
            for (int v : sam.a[u].go)
                if (v) sum[u] += min(sum[v], LLONG_MAX - sum[u]);
        }
    }
```

字典序查询（接上页同一结构体）：

```
    // alphabet must be a permutation of a..z; k is 1-based.
    optional<string> kth(long long k, const string &alphabet = "abcdefghijklmnopqrstuvwxyz") const
    {
        if (k <= 0 || k > sum[0]) return nullopt;
        int u = 0;
        string ans;
        while (k > weight[u])
        {
            k -= weight[u];
            for (char c : alphabet)
            {
                int v = sam.a[u].go[c - 'a'];
                if (!v) continue;
                if (k > sum[v]) k -= sum[v];
                else
                {
                    ans += c;
                    u = v;
                    break;
                }
            }
        }
        return ans;
    }
};
```

应用题补充：Luogu P3975 / 弦论

题意：小写串长至 500000，t=0 时相同子串只计一次，t=1 按出现次数计；1≤k≤10^9。用 counts 作为状态权重，根权重由索引清零；不足输出-1。竞赛应用，不计正式模板题。

所需模板：SuffixAutomaton（第 45 页，6.1 节）、SAMLex（第 50 页，6.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3975

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    int multiple;
    long long k;
    cin >> s >> multiple >> k;
    SuffixAutomaton sam;
    for (char c : s) sam.extend(c - 'a');
    SAMLex index(sam, multiple ? sam.counts() : vector<long long>{});
    auto ans = index.kth(k);
    cout << (ans ? *ans : "-1") << '\n';
}
```

接口用法演示：广义 SAM 查询 / API 示例

演示约定：读 n 个非空小写文档，再读 26 字母排列、1-based k 及非空查询文本 t；输出文档子串并集的第 k 小串（不足-1），然后输出 t 与任一文档 LCS 在 t 内的起点和长度。不计文档重复次数，不求所有文档交集；示例 n=0 合法。

所需模板：GeneralSAM（第 55 页，7.1 节）、SAMLex（第 50 页，6.2 节）、sam_lcs（第 53 页，6.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    GeneralSAM sam;
    for (int i = 0; i < n; i++)
    {
        string s;
        cin >> s;
        sam.add(s);
    }
    sam.build();
    string alphabet, t;
    long long k;
    cin >> alphabet >> k >> t;
    SAMLex index(sam);
    auto ans = index.kth(k, alphabet);
    cout << (ans ? *ans : "-1") << '\n';
    auto [start, length] = sam_lcs(sam, t);
    cout << start << ' ' << length << '\n';
}
```

6.3 SAM 最长公共子串

sam_lcs(sam,t) 接收已有单串 SuffixAutomaton 或已 build 的 GeneralSAM 及小写文本 t，返回 t 中的 {0-based 起点, 长度}；多解取 t 中最早起点，无公共非空串返回 {0,0}。广义 SAM 时求 t 与任一已加入文档的最长公共子串，不是所有文档公共串。t 可空，长度 <INT_MAX；构建 SAM 的成本另计，查询 O(|t|)、额外空间 O(1)，输入与 SAM 不变。沿后缀链接回退并同步匹配长度，不返回源文档坐标；需实际串时用 t.substr(start,length)。

```
// Return {start, length} in t; ties choose the earliest start.
template <class S>
pair<int, int> sam_lcs(const S &sam, const string &t)
{
    int p = 0, length = 0;
    pair<int, int> ans{0, 0};
    for (int i = 0; i < (int)t.size(); i++)
    {
        int c = t[i] - 'a';
        assert(0 <= c && c < 26);
        while (p && !sam.a[p].go[c])
        {
            p = sam.a[p].link;
            length = sam.a[p].len;
        }
        if (sam.a[p].go[c])
        {
            p = sam.a[p].go[c];
            length++;
        }
        else length = 0;
        if (length > ans.second) ans = {i - length + 1, length};
    }
    return ans;
}
```

应用题补充：SPOJ LCS / 最长公共子串

题意：两行小写字符串，各长 ≤250000，输出最长公共子串长度，无公共串输出 0。getline 保留空行，并兼容 CRLF。返回位置属于第二个串，本用法只输出长度；线上 AC 和时间/内存限制待核验。

所需模板：SuffixAutomaton（第 45 页，6.1 节）、sam_lcs（第 53 页，6.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/SP1811

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s, t;
    getline(cin, s);
    getline(cin, t);
    if (!s.empty() && s.back() == '\r') s.pop_back();
    if (!t.empty() && t.back() == '\r') t.pop_back();
    SuffixAutomaton sam;
    for (char c : s) sam.extend(c - 'a');
    cout << sam_lcs(sam, t).second << '\n';
}
```

联合使用示例见第 52 页（广义 SAM 查询 / API 示例），其中直接调用本模板。

第七章　广义后缀自动机

7.1　广义后缀自动机：Trie BFS 构建

先 add(s) 登记若干小写字符串，再 build() 原地将 Trie 变为广义 SAM；build 后禁止继续 add，可重复 build，重新收集串需新建对象。空串与重复串允许，distinct() 返回所有输入串中不同非空子串的总数，不包含跨串拼接。根为 0，a[v].go[c] 为转移、0 表示无边，link 为后缀链接、len 为最长代表串长度；从根沿字符转移可判定子串是否存在。a.size() 为按文档 endpos 等价类构造的状态数，不应直接称为后缀并集语言的最小 DFA 点数；ab、b 是两者不同的例子。P6139 官方样例一要求 endpos 点数 10，后缀语言最小化仅 7 点；该题用 a.size()，勿再最小化。固定 26 字符，设累计输入长度 L、Trie 点数 K，登记总时间 O(L)，build 时间与空间 O(26K)，状态少于 2K；累计输入长度及 2K 须可由 int 表示。克隆只复制已处理转移，不能带入尚未 BFS 处理的 Trie 子边。此版不提供出现次数或各串所属标记。新增 add(p,c) 在 build 前按 Trie 边登记，0<=c<26，p 为已有 Trie 节点；返回子节点号，重复边复用。单次均摊 O(1)，避免把输入树展开成多条长串；add(s) 复用此入口。原 Trie 编号在 build 后仍代表原根路径，但禁止继续 add。

```
struct GeneralSAM
{
    struct Node
    {
        array<int, 26> go{};
        int link = -1, len = 0;
    };

    vector<Node> a{Node{}};
    bool ready = false;

    // Trie step before build; repeated edges return the same node.
    int add(int p, int c)
    {
        assert(!ready && 0 <= p && p < (int)a.size());
        assert(0 <= c && c < 26);
        if (!a[p].go[c])
        {
            int v = a.size();
            a.push_back(Node{});
            a[p].go[c] = v;
        }
        return a[p].go[c];
    }

    void add(const string &s)
    {
        assert(!ready);
        int p = 0;
        for (char ch : s) p = add(p, ch - 'a');
    }
```

BFS 构建（接上页同一结构体）：

```
    void build()
    {
        if (ready) return;
        a.reserve(2 * a.size());
        queue<pair<int, int>> todo;
        for (int c = 0; c < 26; c++)
            if (a[0].go[c]) todo.push({0, c});
        while (!todo.empty())
        {
            auto [parent, c] = todo.front();
            todo.pop();
            int cur = a[parent].go[c], p = a[parent].link;
            a[cur].len = a[parent].len + 1;
            while (p != -1 && !a[p].go[c])
            {
                a[p].go[c] = cur;
                p = a[p].link;
            }
            if (p == -1)
                a[cur].link = 0;
            else
            {
                int q = a[p].go[c];
                if (a[p].len + 1 == a[q].len)
                    a[cur].link = q;
                else
                {
                    int clone = a.size();
                    Node copy = a[q];
                    copy.len = a[p].len + 1;
                    for (int &v : copy.go)
                        if (!a[v].len) v = 0;
                    a.push_back(copy);
                    while (p != -1 && a[p].go[c] == q)
                    {
                        a[p].go[c] = clone;
                        p = a[p].link;
                    }
                    a[q].link = a[cur].link = clone;
                }
            }
            for (int ch = 0; ch < 26; ch++)
                if (int v = a[cur].go[ch]; v && !a[v].len) todo.push({cur, ch});
        }
        ready = true;
    }
```

不同子串计数（接上页同一结构体）：

```
    long long distinct() const
    {
        assert(ready);
        long long ans = 0;
        for (int v = 1; v < (int)a.size(); v++) ans += a[v].len - a[a[v].link].len;
        return ans;
    }
};
```

模板题使用：Library Checker number_of_substrings

题意：本题只有一个字符串，输出不同非空子串数。广义模板可在 build 前多次 add；本例验证其单串特例，多串行为另有独立对拍。若只处理单串，原 SuffixAutomaton 更合适。此题不验证广义 SAM 状态数或最小 DFA 状态数。

所需模板：GeneralSAM（第 55 页，7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/number_of_substrings

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    GeneralSAM sam;
    sam.add(s);
    sam.build();
    cout << sam.distinct() << '\n';
}
```

模板题使用：Luogu P6139

题意：读入所有小写字符串，求不同非空子串数及含根的广义 SAM 点数。先全部 add，再 build，输出 distinct 和 a.size；总长度至 100 万、文档数至 40 万。点数按官方样例的 endpos 等价类口径：aa、ab、bac、caa 输出 10 和 10，不另做后缀语言 DFA 最小化。本例通过完整驱动本地验证，线上待补。

所需模板：GeneralSAM（第 55 页，7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6139

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    GeneralSAM sam;
    while (n--)
    {
        string s;
        cin >> s;
        sam.add(s);
    }
    sam.build();
    cout << sam.distinct() << '\n' << sam.a.size() << '\n';
}
```

接口用法演示：直接 Trie 建图与含空串排名 / API 示例

演示约定：输入 n,q，再给 n 条 parent,c（0≤parent<i），节点 0 为空根；重复同标签边可合并。一次 BFS 构建后输出含空串的不同子串数。每次输入字母序排列和 1-based k，k=1 输出空行，其余调用 kth(k-1)，不足输出-1。自行约定的 API 演示，不冒充 TSUBSTR 官方驱动。

所需模板：GeneralSAM（第 55 页，7.1 节）、SAMLex（第 50 页，6.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    GeneralSAM sam;
    vector<int> state(n + 1);
    for (int i = 1; i <= n; i++)
    {
        int parent;
        char c;
        cin >> parent >> c;
        state[i] = sam.add(state[parent], c - 'a');
    }
    sam.build();
    cout << sam.distinct() + 1 << '\n';
    SAMLex index(sam);
    while (q--)
    {
        string alphabet;
        long long k;
        cin >> alphabet >> k;
        if (k == 1) cout << '\n';
        else
        {
            auto ans = index.kth(k - 1, alphabet);
            cout << (ans ? *ans : "-1") << '\n';
        }
    }
}
```

应用题补充：CodeChef TSUBSTR / Substrings on a Tree

题意：给以 1 为根的无向树及每个顶点的小写字母，求所有祖先到后代路径串中不同串的数量，包含空串；每次按给定 26 字母排列求第 k 小，不足输出-1，k=1 输出空行。N 至 250000、Q 至 50000，k 至 2^63-1，总输出不超过 800KB，原题字母随机。根顶点字母也必须登记；重复标签路径由 Trie 合并。先按树做 BFS 登记，再构建 SAM，不展开根叶字符串。

所需模板：GeneralSAM（第 55 页，7.1 节）、SAMLex（第 50 页，6.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.codechef.com/problems/TSUBSTR

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    string s;
    cin >> n >> q >> s;
    vector<vector<int>> g(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        u--;
        v--;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    GeneralSAM sam;
    vector<int> par(n, -1), state(n);
    state[0] = sam.add(0, s[0] - 'a');
    queue<int> todo;
    todo.push(0);
    while (!todo.empty())
    {
        int u = todo.front();
        todo.pop();
        for (int v : g[u])
        {
            if (v == par[u]) continue;
            par[v] = u;
            state[v] = sam.add(state[u], s[v] - 'a');
            todo.push(v);
        }
    }
```

继续处理操作（接上页同一 main）：

```
    sam.build();
    cout << sam.distinct() + 1 << '\n';
    SAMLex index(sam);
    while (q--)
    {
        string alphabet;
        long long k;
        cin >> alphabet >> k;
        if (k == 1) cout << '\n';
        else
        {
            auto ans = index.kth(k - 1, alphabet);
            cout << (ans ? *ans : "-1") << '\n';
        }
    }
}
```

相关应用：指定状态在线扩展；任意插入顺序不保证 Trie 节点数线性复杂度。

7.2 指定状态在线扩展 SAM

OnlineSAM() 以 0 为根；extend(p,c) 给状态 p 代表的最长串末尾接字符 0<=c<26，返回该串的状态号，调用后立即可沿 go 判断已有子串。旧状态的 len 与编号保持，但 link/go 可变化；不可用短于 len[p] 的匹配串冒充该最长串。逐串输入时每串从 p=0 开始；重复路径允许。total 为当前不同非空子串数，a 含根，公开字段只读；不自动统计出现次数、文档频率或终态。一次至多增加 2 状态，需状态数和长度可用 int 索引，数组可分配。单次 O(当前最大串长 D+26)，Q 次保守 O(QD+26Q)，任意顺序可能平方级；已知整棵 Trie 优先 GeneralSAM 的 BFS 构造。空间 O(26Q)，重置赋新对象。内部递归只进入已有转移分支，深度至多 2；构建完成后可用 SAMLex/sam_lcs，继续扩展前先弃用旧查询索引。

```
struct OnlineSAM
{
    struct Node
    {
        array<int, 26> go{};
        int link = -1, len = 0;
    };

    vector<Node> a{Node{}};
    long long total = 0;
```

指定状态在线扩展（接上页同一结构体）：

```
    // Append c to the longest string represented by state p.
    int extend(int p, int c)
    {
        assert(0 <= p && p < (int)a.size() && 0 <= c && c < 26);
        if (int q = a[p].go[c])
        {
            if (a[q].len == a[p].len + 1) return q;
            int clone = a.size();
            Node copy = a[q];
            copy.len = a[p].len + 1;
            a.push_back(copy);
            a[q].link = clone;
            while (p != -1 && a[p].go[c] == q)
            {
                a[p].go[c] = clone;
                p = a[p].link;
            }
            return clone;
        }
        int cur = a.size();
        Node node;
        node.len = a[p].len + 1;
        a.push_back(node);
        while (p != -1 && !a[p].go[c])
        {
            a[p].go[c] = cur;
            p = a[p].link;
        }
        int parent = p == -1 ? 0 : extend(p, c);
        a[cur].link = parent;
        total += a[cur].len - a[parent].len;
        return cur;
    }
};
```

接口用法演示：在线分支扩展 / API 示例

演示约定：q 次操作，每次给历史版本 parent（0≤parent<i）和小写字符 c；将该版本的完整串接 c 作为新版本，版本 0 为空。保存返回状态号，每次输出当前全部版本的不同非空子串并集大小。这是添加路径，不是持久化 SAM：所有查询读取累计语言。

所需模板：OnlineSAM（第 62 页，7.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    OnlineSAM sam;
    vector<int> state(q + 1);
    for (int i = 1; i <= q; i++)
    {
        int parent;
        char c;
        cin >> parent >> c;
        state[i] = sam.extend(state[parent], c - 'a');
        cout << sam.total << '\n';
    }
}
```

模板题使用：Luogu P6139 / 在线扩展版本

题意：每个文档从根重新扩展，输出不同非空子串数和含根 endpos 状态数；题面最多 40 万串、总长至 100 万。与已有 GeneralSAM 相同的状态口径，不做后缀语言 DFA 最小化；本地验证，线上待补。任意父状态 API 的最坏复杂度不能据此称线性。

所需模板：OnlineSAM（第 62 页，7.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6139

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    OnlineSAM sam;
    for (int i = 0; i < n; i++)
    {
        string s;
        cin >> s;
        int p = 0;
        for (char c : s) p = sam.extend(p, c - 'a');
    }
    cout << sam.total << '\n';
    cout << sam.a.size() << '\n';
}
```

7.3 SAM 文档频率与阈值后缀

代码依赖：Fenwick（第 90 页，C.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

SAMDocuments(sam,s) 接收已完成构建的 GeneralSAM 及同一批小写文档 s；也兼容按该文档集完成的 OnlineSAM 或单文档 SuffixAutomaton。允许空串和重复串，重复输入按不同文档计数。索引期间 sam 须存活且不变。cnt[u] 是该状态任一非空代表串出现的文档数，cnt[0]=s.size() 约定空串出现在每篇文档；不能把单串出现次数当文档数。lengths(k) 要求 k>=1，返回每个状态最长代表串中、出现于至少 k 篇文档的最长后缀长度，无非空解为 0；k 大于文档数全 0。查询任意非空模式串时，从根转移成功后读取 cnt，缺边返回 0。L 为文档总长，V 为状态数，D 为文档数：构造 O((L+V)log(L+2)+D)，空间 O(L+V+D)，每次 lengths 为 O(V)。递归后缀链接树深度不超过最长文档长度加 1；保留递归，不模拟开栈。L、D、V 须小于 INT_MAX，数组可分配；a/g/cnt 视为只读。

```
template <class S>
struct SAMDocuments
{
    const S &sam;
    vector<vector<int>> g;
    vector<int> cnt;
```

文档标记与子树去重（接上页同一结构体）：

```
    SAMDocuments(const S &sam, const vector<string> &s)
        : sam(sam), g(sam.a.size()), cnt(sam.a.size())
    {
        int n = sam.a.size(), size = 0;
        vector<vector<int>> tag(n);
        for (int u = 1; u < n; u++) g[sam.a[u].link].push_back(u);
        for (int id = 0; id < (int)s.size(); id++)
        {
            int p = 0, len = 0;
            for (char c : s[id])
            {
                assert('a' <= c && c <= 'z');
                p = sam.a[p].go[c - 'a'];
                len++;
                assert(p && sam.a[p].len == len);
                tag[p].push_back(id);
                size++;
            }
        }
        Fenwick<int> bit(size);
        vector<int> last(s.size());
        int timer = 0;
        auto dfs = [&](auto &&self, int u) -> void
        {
            int l = timer + 1;
            for (int id : tag[u])
            {
                if (last[id]) bit.add(last[id], -1);
                last[id] = ++timer;
                bit.add(timer, 1);
            }
            for (int v : g[u]) self(self, v);
            cnt[u] = bit.query(l, timer);
        };
        dfs(dfs, 0);
        cnt[0] = s.size();
    }
```

阈值后缀长度（接上页同一结构体）：

```
    // Longest suffix length occurring in at least k documents, for each state.
    vector<int> lengths(int k) const
    {
        assert(k >= 1);
        vector<int> best(cnt.size());
        auto dfs = [&](auto &&self, int u) -> void
        {
            if (cnt[u] >= k) best[u] = sam.a[u].len;
            for (int v : g[u])
            {
                best[v] = best[u];
                self(self, v);
            }
        };
        dfs(dfs, 0);
        return best;
    }
};
```

应用题补充：Codeforces 204E / Little Elephant and Strings

题意：给 n 篇非空小写文档和 k，分别求每篇中出现于至少 k 篇文档的子串区间个数；重复文档分别计数，区间重复出现也分别计数。n、k 及总长度至 100000，k 允许大于 n。先统计文档频率，再按每个前缀累加合格最长后缀长度；答案用 long long。比赛应用，不登记为非比赛模板题。

所需模板：GeneralSAM（第 55 页，7.1 节）、Fenwick（第 90 页，C.1 节）、SAMDocuments（第 65 页，7.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/problemset/problem/204/E

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    vector<string> s(n);
    GeneralSAM sam;
    for (auto &t : s)
    {
        cin >> t;
        sam.add(t);
    }
    sam.build();
    SAMDocuments index(sam, s);
    auto best = index.lengths(k);
    for (int id = 0; id < n; id++)
    {
        int p = 0;
        long long ans = 0;
        for (char c : s[id])
        {
            p = sam.a[p].go[c - 'a'];
            ans += best[p];
        }
        cout << ans << (id + 1 == n ? '\n' : ' ');
    }
}
```

接口用法演示：子串文档频率 / API 示例

演示约定：输入 n,q、n 篇非空小写文档和 q 个非空小写模式串，输出每个模式出现于多少篇输入文档；同一文档内重复出现只算一次，相同输入文档分别计数。走转移失败立即返回 0。本例 n 可为 0；核心另支持空文档，空串频率为 cnt[0]。

所需模板：GeneralSAM（第 55 页，7.1 节）、Fenwick（第 90 页，C.1 节）、SAMDocuments（第 65 页，7.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<string> s(n);
    GeneralSAM sam;
    for (auto &t : s)
    {
        cin >> t;
        sam.add(t);
    }
    sam.build();
    SAMDocuments index(sam, s);
    while (q--)
    {
        string t;
        cin >> t;
        int p = 0;
        for (char c : t)
        {
            p = sam.a[p].go[c - 'a'];
            if (!p) break;
        }
        cout << (p ? index.cnt[p] : 0) << '\n';
    }
}
```

第八章　Manacher

8.1　Manacher 回文半径

manacher(s) 返回 odd/even，下标从 0 开始。odd[i] 包含中心，闭区间 [i-odd[i]+1,i+odd[i]-1]，长度 2*odd[i]-1；even[i] 的中心在 i 前，区间 [i-even[i],i+even[i]-1]，长度 2*even[i]，零半径为空。空串合法，支持任意字节（含零），按字节而非 Unicode 字素比较；int 索引要求 n<2^30。最长回文取上述长度最大值；全部 odd/even 以 long long 求和得到回文出现总数。O(n) 时间和空间，边界显式检查，无需选择哨兵字符。输入为 string、vector<int> 或 vector<long long> 对象，支持空序列和任意整数；泛型 S 需 size()、随机下标及 O(1) 相等比较。字符串字面量请显式写 string("abc")。输入不拷贝，输出移动返回。

```
// odd[i] includes the center; even[i] is centered before i.
template <class S>
pair<vector<int>, vector<int>> manacher(const S &s)
{
    int n = (int)s.size();
    vector<int> odd(n), even(n);
    for (int i = 0, l = 0, r = -1; i < n; i++)
    {
        int k = i > r ? 1 : min(odd[l + r - i], r - i + 1);
        while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) ++k;
        odd[i] = k--;
        if (i + k > r)
        {
            l = i - k;
            r = i + k;
        }
    }
    for (int i = 0, l = 0, r = -1; i < n; i++)
    {
        int k = i > r ? 0 : min(even[l + r - i + 1], r - i + 1);
        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) ++k;
        even[i] = k--;
        if (i + k > r)
        {
            l = i - k - 1;
            r = i + k;
        }
    }
    return {move(odd), move(even)};
}
```

模板题使用：Library Checker enumerate_palindromes

题意：给非空字符串，从左到右输出每个字符及相邻字符间隙处的最长回文长度，共 2n-1 项。odd 含中心，even[i] 的中心在 i 前面，因此分别转成 2*odd[i]-1 和 2*even[i]。

所需模板：manacher（第 70 页，8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/enumerate_palindromes

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    int n = s.size();
    auto [odd, even] = manacher(s);
    vector<int> answer(2 * n - 1);
    for (int i = 0; i < n; i++)
    {
        answer[2 * i] = 2 * odd[i] - 1;
        if (i) answer[2 * i - 1] = 2 * even[i];
    }
    for (int i = 0; i < (int)answer.size(); i++)
        cout << answer[i] << (i + 1 == (int)answer.size() ? '\n' : ' ');
}
```

模板题使用：Luogu P3805

题意：给定长度不超过 11000000 的小写字符串，输出最长回文子串长度。odd 含中心，even[i] 以 i 前间隙为中心，分别用 2*odd[i]-1 和 2*even[i] 取最大值。

所需模板：manacher（第 70 页，8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3805

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    auto [odd, even] = manacher(s);
    int answer = 0;
    for (int i = 0; i < (int)s.size(); i++)
    {
        answer = max(answer, 2 * odd[i] - 1);
        answer = max(answer, 2 * even[i]);
    }
    cout << answer << '\n';
}
```

接口用法演示：整数序列回文半径 / API 示例

演示约定：输入 n 及 n 个 signed64 整数，输出两行 odd/even 半径，均 0-based；odd 含中心，even 中心在 i 之前。n=0 输出两空行。接口示例，不对应正式模板题或线上 AC。

所需模板：manacher（第 70 页，8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    auto [odd, even] = manacher(a);
    for (int x : odd) cout << x << ' ';
    cout << '\n';
    for (int x : even) cout << x << ' ';
    cout << '\n';
}
```

第九章　回文树

9.1　回文树 PAM：回文后缀与出现次数

add(ch) 追加小写英文字母，返回最长回文后缀节点，亦保存于 last。节点 0 是长度 0 根，1 是长度 -1 根，实际回文节点从 2 开始；distinct() 为不同非空回文串数。t[u].len 为长度，fail 为最长真回文后缀链接，suffixes 为该节点的非空回文后缀数量；追加后的 t[last].suffixes 就是本位置结尾的回文串数。total 为当前全部回文子串的出现次数之和。end 为首次出现时的 0-based 右端点，可恢复原串子段；hits 仅统计作为最长后缀的次数，不能当出现次数。occurrences() 返回按节点编号的完整出现次数副本，根置零，O(n)；重复调用以及查询后继续追加均安全。重新赋值 PalindromicTree() 清空重建。固定 26 字符集下建树 O(n)，空间 O(26n)。

```
struct PalindromicTree
{
    struct Node
    {
        int next[26]{};
        int len = 0, fail = 0, suffixes = 0, end = -1;
        long long hits = 0;
    };

    vector<Node> t;
    vector<int> s;
    int last = 0;
    long long total = 0;

    PalindromicTree() : t(2), s(1, -1)
    {
        t[0].fail = 1;
        t[1].len = -1;
    }

    int get_fail(int u) const
    {
        int pos = s.size() - 1;
        while (s[pos - t[u].len - 1] != s[pos]) u = t[u].fail;
        return u;
    }
```

接上页同一结构体：

```
    // Append a lowercase letter; return the longest palindromic suffix node.
    int add(char ch)
    {
        int c = ch - 'a';
        assert(0 <= c && c < 26);
        s.push_back(c);
        int u = get_fail(last);
        if (!t[u].next[c])
        {
            int v = t.size();
            t.push_back(Node{});
            t[v].len = t[u].len + 2;
            t[v].fail = t[get_fail(t[u].fail)].next[c];
            t[v].suffixes = t[t[v].fail].suffixes + 1;
            t[v].end = (int)s.size() - 2;
            t[u].next[c] = v;
        }
        last = t[u].next[c];
        t[last].hits++;
        total += t[last].suffixes;
        return last;
    }

    int distinct() const { return (int)t.size() - 2; }

    // Copy-and-propagate: repeated calls and later appends remain valid.
    vector<long long> occurrences() const
    {
        vector<long long> count(t.size());
        for (int u = 2; u < (int)t.size(); u++) count[u] = t[u].hits;
        for (int u = (int)t.size() - 1; u >= 2; u--) count[t[u].fail] += count[u];
        count[0] = 0;
        count[1] = 0;
        return count;
    }
};
```

模板题使用：Luogu P5496

题意：按上一次答案把当前字符循环偏移后追加，输出以当前位置结尾的回文串个数。add 返回最长回文后缀节点，读取其 suffixes。

所需模板：PalindromicTree（第 73 页，9.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5496

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::string s;
    std::cin >> s;
    PalindromicTree pam;
    int answer = 0;
    for (int i = 0; i < (int)s.size(); i++)
    {
        char ch = 'a' + (s[i] - 'a' + answer) % 26;
        int u = pam.add(ch);
        answer = pam.t[u].suffixes;
        std::cout << answer << (i + 1 == (int)s.size() ? '\n' : ' ');
    }
}
```

应用题补充：Luogu P3649

题意：求所有回文子串中“长度乘出现次数”的最大值。整串追加完后调用 occurrences 汇总次数，不能直接把 hits 当完整出现次数。

所需模板：PalindromicTree（第 73 页，9.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3649

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::string s;
    std::cin >> s;
    PalindromicTree pam;
    for (char ch : s) pam.add(ch);
    auto count = pam.occurrences();
    long long answer = 0;
    for (int u = 2; u < (int)count.size(); u++)
        answer = std::max(answer, count[u] * pam.t[u].len);
    std::cout << answer << '\n';
}
```

第十章　最小表示法

10.1　最小循环表示

minimum_rotation(s) 返回字典序最小循环表示的起点 (0-based)，空串返回 0；按无符号字节比较，与 string 字典序一致。O(n)。

```
int minimum_rotation(const string &s)
{
    int n = (int)s.size(), i = 0, j = 1, k = 0;
    if (!n) return 0;
    while (i < n && j < n && k < n)
    {
        unsigned char a = s[(i + k) % n], b = s[(j + k) % n];
        if (a == b)
        {
            ++k;
            continue;
        }
        if (a > b)
        {
            i += k + 1;
            if (i == j) ++i;
        }
        else
        {
            j += k + 1;
            if (i == j) ++j;
        }
        k = 0;
    }
    return min(i, j);
}
```

模板题使用：Luogu P13270【模板】最小表示法

题意：给定长度 1..10^7 的小写字母字符串，输出所有循环位移中字典序最小的字符串。minimum_rotation 返回 0-based 起点，使用 rotate 原地重排后输出；O(n) 时间，算法本体 O(1) 额外空间。

所需模板：minimum_rotation（第 76 页，10.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P13270

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    string s;
    if (!(cin >> n >> s)) return 0;
    int p = minimum_rotation(s);
    rotate(s.begin(), s.begin() + p, s.end());
    cout << s << '\n';
}
```

第十一章　Lyndon 分解

11.1　Lyndon 分解：Duval

lyndon(s) 返回严格递增的因子右端点 ends；按 0-based 半开区间 [0,ends[0])、[ends[i-1],ends[i]) 取因子，数值恰好也是 1-based 闭区间右端点。各因子严格小于自己的所有非空真后缀，因子序列按字典序非递增，分解唯一。输入按 unsigned char 的 0..255 字节序比较，支持空串、零字节与高位字节；不是 Unicode 字符顺序。空串返回空 vector，s 不变，长度须 ≤INT_MAX。Duval 时间 O(n)，预留 n 个 int 的输出容量，空间 O(n)，除返回数组外只需 O(1) 工作空间；不复制子串。P6114 直接异或 ends，无需再加 1；全部相同字符会拆成 n 个单字符，不能合并相等因子。

```
vector<int> lyndon(const string &s)
{
    assert(s.size() <= INT_MAX);
    int n = s.size();
    vector<int> ends;
    ends.reserve(n);
    int i = 0;
    while (i < n)
    {
        int j = i + 1, k = i;
        while (j < n && (unsigned char)s[k] <= (unsigned char)s[j])
        {
            if (s[k] == s[j])
                k++;
            else
                k = i;
            j++;
        }
        int len = j - k;
        while (i <= k)
        {
            i += len;
            ends.push_back(i);
        }
    }
    return ends;
}
```

模板题使用：P6114 【模板】Lyndon 分解

题意：输入小写字符串，输出非递增 Lyndon 分解的全部 1-based 右端点异或和；n≤5000001。

所需模板：lyndon（第 78 页，11.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6114

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    int ans = 0;
    for (int r : lyndon(s))
        ans ^= r;
    cout << ans << '\n';
}
```

接口用法演示：API 字节串完整分解

演示约定：输入 n 和 n 个 0..255 整数作为字节串，可为空；输出因子个数及全部右端点，按 unsigned char 字节序。

所需模板：lyndon（第 78 页，11.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/string/lyndon/

```
int main()
{
    int n;
    cin >> n;
    string s(n, '\0');
    for (int i = 0; i < n; i++)
    {
        int x;
        cin >> x;
        s[i] = (char)x;
    }
    auto ends = lyndon(s);
    cout << ends.size() << '\n';
    for (int r : ends)
        cout << r << ' ';
    cout << '\n';
}
```

第十二章　配套知识与验证范围

12.1　回文半径与在线回文计数

Manacher 的 odd[i] 包含中心，最长奇回文为 $[i-odd[i]+1, i+odd[i]-1]$，长度 $2odd[i]-1$；even[i] 以 i 前间隙为中心，最长偶回文为 $[i-even[i], i+even[i]-1]$，长度 $2even[i]$。所有 odd/even 之和是回文的出现总次数，应以 long long 累加；这不是不同回文串数量。恢复最长子串时记录对应起点和长度即可。

附件插入分隔符后的半径有包含中心与不含中心两种约定，不能直接套到本库数组。哨兵版本还依赖字母表：小写字符串的写法不能原样推广到含标记或零字节的输入。本库显式检查边界，空串和任意字节均合法；按字节而非 Unicode 字素比较，int 索引要求 $n < 2^{30}$。P3805 的 1100 万字符相同、交替和无长度至少二回文的三种闭式输入已通过双模式本地检查。

PAM 的 suffixes 是当前回文节点的非空回文后缀数量，等于 fail 节点的 suffixes 加一。P5496 必须先按上一个答案解码当前字符，再 add，输出 t[last].suffixes。hits 仅统计作为最长后缀的命中次数；完整次数应取 occurrences() 的返回值。该函数复制后再沿 fail 汇总，不改 hits，因此重复查询及查询后继续追加均安全。total 为逐位置 suffixes 的总和，最多 $n(n+1)/2$；distinct 才是不同非空回文串数。PAM 固定小写字母，内部前置 -1 与 $0 \ldots 25$ 编码不同，不继承 Manacher 的任意字节契约。

独立逐子串逆序比较、PAM 全短串及两题最大规模驱动均在普通和 ASan/UBSan 下通过。Manacher 的 LC 全部 24 组官方数据双模式本地 checker 接受，不等于线上 AC。详细接口与记录见仓库 docs/PALINDROME-ATTACHMENT.md。

12.2　整数后缀数组的边界与 SAM 次数

SuffixArray 的整数入口要求 $0 \le a[i] < alphabet$；稀疏大整数先保序离散化。sa/rk/lcp 都从零编号，lcp[k] 比较 sa[k-1] 与 sa[k]，首项为零。附件第二份倍增模板额外要求 LCP 遇分隔符停止，不能把普通 LCP 不加处理地代入。令 remain[i] 为从 i 到下一个分隔符之前的普通字符数，则分段前缀长为

$$height[k] = \min\{lcp[k], remain[sa[k-1]], remain[sa[k]]\}.$$

分隔符位置 remain 为零，连续或重复分隔符同样适用；保留原 lcp 可避免两种语义混用。两个字符串拼接后，相同前缀的后缀在 SA 中连续，所以找最长公共子串只需检查异来源的相邻后缀。输出时转回各自原串的半开区间，不把分隔符当原字符。

SAM 状态 u 代表长度 $(len[link[u]], len[u]]$ 内具有同一 endpos 集合的子串。真实扩展状态的 occ 种子为一，克隆为零；附件存放插入 id 的 endpos 字段不等于完整出现次数。counts 按长度倒序传播，不能按创建编号倒序，因为克隆可能后创建。查询返回副本，不改种子，所以重复查询后继续 extend 安全。根计数为串长 n，不能解释成空串的 $n+1$ 个出现位置；非空子串只看非根状态。P3804 仅在 count[u]>1 时取 count[u]*len[u]，乘积用 long long，无重复子串则为零。若需要每个前缀对应的状态，调用者在每次 extend 后保存 last 即可。

独立后缀排序、逐子串 endpos 集合、分隔符截断与完整百万规模驱动双模式通过。LC 后缀数组 50 组、最长公共子串 28 组官方数据双模式本地 checker 接受，不等于线上 AC 或速度排名。细节见仓库 docs/SUFFIX-ATTACHMENT.md。

12.3　广义 SAM 点数与后缀语言

GeneralSAM 的状态按文档编号与结束位置组成的 endpos 集合等价。先 add 全部小写串，再 build；根为零，distinct 返回不同非空子串总数，a.size() 包含根。构造后不再 add，重复 build 安全；重建另开对象。重复串和空串在库内合法，P6139 则要求输入非空串。

多串 endpos 类不能直接称为后缀并集的最小 DFA。例如 ab、b 的三个非空子串 a、b、ab 具有不同 endpos 集合，含根共四点；仅识别后缀并集时，读完 b 和 ab 的状态可合并，含初态只需三点（省略拒绝汇点）。P6139 官方样例 aa、ab、bac、caa 要求不同子串数与点数均为十，符合 endpos 口径；把后缀 Trie 按右语言最小化则为七点，不能据题面中的“最小”二字输出七。完整用法直接输出 a.size()，不额外做 DFA 合并。

独立枚举文档位置集合与两种右语言计算核对上述区别。完整驱动的 600 组随机参考及排列、百万字符链和克隆密集串、两条各五十万字符的交替串、四十万文档合计百万字符，以及深十五完整二叉 Trie 均通过普通和 ASan/UBSan。这些是本地证据，尚未新增 P6139 线上 AC 或排名。构造与口径详见仓库 docs/GENERAL-SAM-NOTES.md。

第一部分

跨类依赖

附录 A　数论

A.1　模算术简介

A.1.1　静态模整数

固定 int 模数 1≤mod≤INT_MAX，可为合数。构造接受 signed64 并正规化，v 保持 0..mod-1；不要手动破坏正规化。加减乘 O(1)，pow 非负 signed64 指数、时间 O(log(e+1))。try_inv 用欧几里得求逆：gcd(v,mod)=1 时返回 optional 值，否则 nullopt；mod=1 返回代表 0。inv、/、/= 要求分母可逆；断言开启时诊断违反前提，关闭断言不承诺拒绝非法除法。逆元 O(log(mod+1))、额外空间 O(1)，只用 long long，不新增辅助模板依赖。+=、-=、*=、/= 返回自身引用，支持自赋值与链式运算，可作为 Fenwick 区间和值类型。类型的扩展不放宽 Binomial、NTT、高斯或 FPS 自身的素数等前提。

```
template <int mod> struct ModInt
{
    static_assert(mod >= 1);
    int v;

    ModInt(long long x = 0)
    {
        v = x % mod;
        if (v < 0)
            v += mod;
    }

    ModInt operator+(ModInt b) const
    {
        long long x = (long long)v + b.v;
        if (x >= mod)
            x -= mod;
        ModInt r;
        r.v = x;
        return r;
    }

    ModInt operator-(ModInt b) const
    {
        int x = v - b.v;
        if (x < 0)
            x += mod;
        ModInt r;
        r.v = x;
        return r;
    }

    ModInt operator*(ModInt b) const
    {
        ModInt r;
        r.v = 1LL * v * b.v % mod;
        return r;
    }
```

模整数（接上页同一结构体）：

```
    ModInt pow(long long e) const
    {
        assert(e >= 0);
        ModInt a = *this, r = 1;
        while (e)
        {
            if (e & 1)
                r = r * a;
            a = a * a;
            e >>= 1;
        }
        return r;
    }

    optional<ModInt> try_inv() const
    {
        long long a = v, b = mod, x = 1, y = 0;
        while (b)
        {
            long long q = a / b;
            a -= q * b;
            swap(a, b);
            x -= q * y;
            swap(x, y);
        }
        if (a != 1)
            return nullopt;
        return ModInt(x);
    }

    ModInt inv() const
    {
        auto r = try_inv();
        assert(r);
        return *r;
    } // requires an invertible residue
```

模整数（接上页同一结构体）：

```
    ModInt operator/(ModInt b) const
    {
        return *this * b.inv();
    }

    ModInt &operator+=(ModInt b)
    {
        return *this = *this + b;
    }

    ModInt &operator-=(ModInt b)
    {
        return *this = *this - b;
    }

    ModInt &operator*=(ModInt b)
    {
        return *this = *this * b;
    }

    ModInt &operator/=(ModInt b)
    {
        return *this = *this / b;
    }
};
```

联合使用示例见第 89 页（Luogu P1313 计算系数），其中直接调用本模板。

A.2 卢卡斯定理

A.2.1 Lucas：小素数模的大下标组合数

Lucas(p) 预处理素数模数 p，choose(n,k) 返回 C(n,k) 模 p；n、k 为完整 unsigned long long 范围，k>n 返回 0，C(0,0)=1。重新赋值 Lucas(p) 可更换模数并重新预处理；不检测素性，要求调用者保证 p 为素数且 p≥2。预处理时间 O(p+log p)，动态版空间 O(p)；单次查询 O(1+log_p(n+1))。适合小素数模数，通常 $p \le 10^6$；大素数不能因查询下标小就忽略这里整段 O(p) 的预处理。fac、ifac 下标为 0..p-1，模数和表只读；中间乘法用 long long。合数模数请使用 exLucas。

```
struct Lucas
{
    int mod;
    vector<int> fac, ifac;

    Lucas(int p)
    {
        assert(p >= 2);
        mod = p;
        fac.resize(p);
        ifac.resize(p);
        fac[0] = 1;
        for (int i = 1; i < p; i++) fac[i] = 1LL * fac[i - 1] * i % p;
        long long a = fac[p - 1], r = 1;
        for (int e = p - 2; e; e >>= 1)
        {
            if (e & 1) r = r * a % p;
            a = a * a % p;
        }
        ifac[p - 1] = r;
        for (int i = p - 1; i; i--) ifac[i - 1] = 1LL * ifac[i] * i % p;
    }
```

```
    int choose(unsigned long long n, unsigned long long k) const
    {
        if (k > n) return 0;
        long long answer = 1;
        while (n || k)
        {
            int a = n % mod, b = k % mod;
            if (b > a) return 0;
            answer = answer * fac[a] % mod;
            answer = answer * ifac[b] % mod;
            answer = answer * ifac[a - b] % mod;
            n /= mod;
            k /= mod;
        }
        return answer;
    }
};
```

模板题使用：Luogu P3807

题意：多组查询 C(n+m,n) 模 p，题目保证 p 为素数且不超过 10^5。每组构造 Lucas(p)，choose 的参数为 n+m、n；此题范围内 n+m 不会溢出。

所需模板：Lucas（第 86 页，A.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3807

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        unsigned long long n, m;
        int p;
        cin >> n >> m >> p;
        Lucas solver(p);
        cout << solver.choose(n + m, n) << '\n';
    }
    return 0;
}
```

附录 B　组合数学

B.1　排列组合

B.1.1　阶乘组合数

代码依赖：ModInt（第 83 页，A.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

Binomial<prime>(n=0) 预处理 0..n，要求 prime 为素数且 0≤n<prime。init(n) 只扩大当前上限，较小或相同的 n 不改变表；choose(n,k) 返回 C(n,k)，permute(n,k) 返回 n!/(n-k)!。查询前显式预处理到 n，查询本身不扩表。n<0、k<0 或 k>n 返回 0，C(0,0)=P(0,0)=1。fac、ifac 为可直接读取的阶乘与逆阶乘表，增长可能使旧引用失效。单次查询 O(1)；从 old 扩到 n 新增 O(n-old+log prime) 次域运算，重分配还可能复制旧表。首次预处理 O(n+log prime)，空间 O(n)。超过模数不能使用阶乘除法：小素数用 Lucas，小合数用 exLucas。

```
template <int mod> struct Binomial
{
    using Z = ModInt<mod>;
    vector<Z> fac{Z(1)}, ifac{Z(1)};

    Binomial(int n = 0) { init(n); }

    void init(int n)
    {
        assert(0 <= n && n < mod);
        int old = (int)fac.size() - 1;
        if (n <= old) return;
        fac.resize(n + 1);
        ifac.resize(n + 1);
        for (int i = old + 1; i <= n; i++) fac[i] = fac[i - 1] * i;
        ifac[n] = fac[n].inv();
        for (int i = n; i > old; i--) ifac[i - 1] = ifac[i] * i;
    }
```

```
    Z choose(int n, int k) const
    {
        if (k < 0 || k > n) return 0;
        assert(n < (int)fac.size());
        return fac[n] * ifac[k] * ifac[n - k];
    }

    Z permute(int n, int k) const
    {
        if (k < 0 || k > n) return 0;
        assert(n < (int)fac.size());
        return fac[n] * ifac[n - k];
    }
};
```

应用题补充：Luogu P1313 计算系数

题意：求 (by+ax) 的 k 次方中指定项的系数，模 10007。x、y 的指数分别为 n、m，均非负且 n+m=k。k 至多 1000，a、b 均为不超过 1000000 的非负数。组合 C(k,n) 与两个幂即可。模数为素数且 k 小于模数，满足阶乘逆元前提；零次幂按空乘积为 1。

所需模板：ModInt（第 83 页，A.1.1 节）、Binomial（第 88 页，B.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1313

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int a, b, k, n, m;
    cin >> a >> b >> k >> n >> m;
    Binomial<10007> c(k);
    auto x = ModInt<10007>(a).pow(n);
    auto y = ModInt<10007>(b).pow(m);
    cout << (c.choose(k, n) * x * y).v << '\n';
}
```

附录 C　树状数组

C.1　树状数组第 k 小与组合依赖

保留频数第 k 小和泛型组合用途；P3374 基础单点加/区间和题不再独立收录或安排提交。add(x,v)、sum(x)、query(l,r)，下标 1..n，闭区间；kth(k) 要求所有频率非负且 k>0，超过总和返回 n+1。单次 O(log n)。T 须能从 0 构造并支持 += 与减法；可直接使用 ModInt 维护模数下的区间和。kth 只适用于有序累计频数，不适用于取模后的区间和。

```
template <class T = long long> struct Fenwick
{
    int n;
    vector<T> a;

    Fenwick(int n = 0) : n(n), a(n + 1) {}

    void add(int x, T v)
    {
        assert(x >= 1);
        for (; x <= n; x += x & -x) a[x] += v;
    }

    T sum(int x) const
    {
        T ans = 0;
        for (; x; x -= x & -x) ans += a[x];
        return ans;
    }

    T query(int l, int r) const { return sum(r) - sum(l - 1); }

    // Nonnegative frequencies; returns n+1 if k exceeds total.
    int kth(T k) const
    {
        assert(k > 0);
        int x = 0, b = 1;
        while (b <= n / 2) b *= 2;
        for (; b; b >>= 1)
            if (x + b <= n && a[x + b] < k)
            {
                k -= a[x + b];
                x += b;
            }
        return x + 1;
    }
};
```

模板题使用：Luogu P3369

题意：维护可重集合的插入、删除一个、排名、第 k 小和严格前驱/后继。预读全部操作并离散化非第 k 小查询的值域；频数下标为 1-based，kth 的返回值减 1 后映射回原值。排名查询允许 x 不在集合中；第 k 小与前驱/后继按题目保证存在。n≤100000，|x|≤10000000；总复杂度 O(n log n)，空间 O(n)。

所需模板：Fenwick（第 90 页，C.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3369

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<pair<int, int>> q(n);
    vector<int> a;
    for (auto &[op, x] : q)
    {
        cin >> op >> x;
        if (op != 4) a.push_back(x);
    }
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    Fenwick<int> f(a.size());
    for (auto [op, x] : q)
    {
        int p = lower_bound(a.begin(), a.end(), x) - a.begin();
        if (op == 1)
            f.add(p + 1, 1);
        else if (op == 2)
        {
            if (f.query(p + 1, p + 1) > 0) f.add(p + 1, -1);
        }
        else if (op == 3)
            cout << f.sum(p) + 1 << '\n';
        else if (op == 4)
            cout << a[f.kth(x) - 1] << '\n';
        else if (op == 5)
            cout << a[f.kth(f.sum(p)) - 1] << '\n';
        else
        {
            p = upper_bound(a.begin(), a.end(), x) - a.begin();
            cout << a[f.kth(f.sum(p) + 1) - 1] << '\n';
        }
    }
}
```

算法索引

ac_shortest, 22
ACWeighted, 24
AhoCorasick, 16

Binomial, 88

DC3, 40
DynamicAC, 26

exkmp, 14

Fenwick, 90

GeneralSAM, 55
good_number_sum, 12

kmp_match, 7

Lucas, 86
lyndon, 78

manacher, 70
minimum_rotation, 76
ModInt, 83

OnlineSAM, 62
order_match, 9

PalindromicTree, 73
prefix_function, 5
prefix_lcs, 38

SAIS, 28
sam_lcs, 53
SAMDocuments, 65
SAMLex, 50
square_counts, 38
StringHash, 1
SuffixArray, 32
SuffixAutomaton, 45
SuffixLCP, 35

z_function, 11